

STAT395: Sampling

Richard Arnold et al.

2026-04-11

Contents

1	Introductory Remarks	7
1.1	Lecture Plan	7
1.2	Recommended reading	8
1.3	R Packages	9
2	Sampling	11
2.1	Random numbers	11
2.2	Sampling from categorical random variables	15
2.3	Sampling from discrete numerical random variables	20
2.4	Sampling from continuous random variables	24
2.5	Sampling from a finite collection	29
3	Sample Surveys	35
3.1	The Survey Process	35
3.2	Populations and Frames	37
3.3	Approaches to sampling	40
3.4	Example datasets	40
4	Statistical Inference	43
4.1	Inference in infinite populations	43
4.2	Inference in finite populations	47
4.3	The survey inference process	49
4.4	Example	51
5	Sample Designs	53
5.1	Census	53
5.2	Simple Random Sampling Without Replacement (SRSWOR)	53
5.3	Simple Random Sampling With Replacement (SRSWR)	57
5.4	Probability proportion to size Sampling (PPSWR)	58
5.5	Linear Systematic Random Sample (LSRS)	60
5.6	Stratified Simple Random Sample (STSRs)	63
5.7	One stage cluster sampling (S1C)	68
5.8	Two stage cluster sampling (S2C)	68
5.9	Complex multistage designs	68
5.10	Introduction to the design effect	68
6	Inference with R	71
7	Missing data	73
	References	75

A Computing the Required Sample Size**77**`library(haven)``library(dplyr)``##``## Attaching package: 'dplyr'``## The following objects are masked from 'package:stats':``##``## filter, lag``## The following objects are masked from 'package:base':``##``## intersect, setdiff, setequal, union``library(ggplot2)``library(kableExtra)``##``## Attaching package: 'kableExtra'``## The following object is masked from 'package:dplyr':``##``## group_rows``library(survey)``## Loading required package: grid``## Loading required package: Matrix``## Loading required package: survival``##``## Attaching package: 'survey'``## The following object is masked from 'package:graphics':``##``## dotchart``library(srvyr)``##``## Attaching package: 'srvyr'``## The following object is masked from 'package:kableExtra':``##``## group_rows``## The following object is masked from 'package:stats':``##``## filter``library(sampling)``##``## Attaching package: 'sampling'``## The following objects are masked from 'package:survival':``##`

```
##      cluster, strata
```

```
#library(gtsummary) # error installing/loading this  
library(mice)
```

```
##
```

```
## Attaching package: 'mice'
```

```
## The following object is masked from 'package:stats':
```

```
##
```

```
##      filter
```

```
## The following objects are masked from 'package:base':
```

```
##
```

```
##      cbind, rbind
```

```
#library(simputation) # error installing/loading this  
library(scales)  
source("utils.R")
```


Chapter 1

Introductory Remarks

These are notes for the Sampling section of STAT395/455. The course overlaps to some extent with STAT392/439, but there is a greater emphasis on R coding, computation, summary and display.

1.1 Lecture Plan

Course Learning Objectives

- Describe the steps in the survey process, including the sources of survey error;
- Draw samples from a sample frame under a variety of sample designs, and compute survey weights;
- Carry out, interpret and present statistical inference procedures on survey data, including adjustments for non-response.

Lecture plan (3 lectures + 1 tutorial per week)

Bring a laptop to the Friday (tutorial) lecture.

Week 1 – The Survey Process

- The survey process, sources of survey error
- Populations and Frames
- Sampling in finite and infinite populations; Basics of SRSWOR inference; sample weights;
- **Tutorial:** Sampling basics; Population and Frames

Week 2 – Sampling

Use of `sample()` and other simulation methods in R for a variety of designs [`sampler` package (does power calculations, allocation, SRS and STSRS, but not cluster)] [`sampling` package – can do cluster samples]

- Simple random sampling, with and without replacement
- Stratified sampling
- Cluster sampling
- Sampling in complex designs
- Sample size calculation and allocation
- Survey weights – create estimates from weights without standard errors

- **Tutorial:** using R to draw samples and compute survey weights

Week 3 - Inference with the R survey package

- Inference in simple and complex designs, including regression
- Variance estimation
- Definition and interpretation of the design effect
- **Tutorial:** using R to specify designs and draw inferences

Week 4 – Non-response and complex designs

- Post-stratification for non-response adjustment
- Imputation (`simputation` package)
- Jackknife and bootstrap inference
- **Tutorial:** raking adjustments to weights; imputation; jackknife inference

Assignment 4 (10%) [due end of Week 12: covers material from Weeks 9-11]

- Survey process, populations and frames
- Describe a real survey, comment on design
- Propose a sample design, frame, comment on scope and coverage
- Draw and analyse a sample under various designs
- Compute sample sizes and allocate samples in stratified sampling
- Compute and interpret the design effect
- Analyse data and present results from a survey

Test 2 (30%) [assessment period] [1.5 hours, closed book]

- Survey process, populations and frames
- Describe sample designs
- Compute and interpret sample weights
- Use sample weights for point estimates
- Interpret output from survey inference in R
- Compare results, interpret design effect
- Describe methods for non-response adjustment (imputation and post-stratification); comment on bias reduction and assumptions being made
- Describe the jackknife method of variance estimation

1.2 Recommended reading

There are lots of books on sample surveys.

- Thomas Lumley’s book ‘Complex Surveys’ (Lumley (2010)) is a good introduction to sample survey analysis with R, and accompanies the **survey** package (Lumley (2004)).
- Groves et al. ‘Survey Methodology’ (groves.etal) is a good general summary of the important issues in survey design.
- Sarndal, Swensson and Wretman is a classic in the field of the theory of sampling (Särndal et al. (1992))
- Likewise the books by Cochran (1977) and Kish (1965)
- Little and Rubin have a great book on missing data (Little and Rubin (2002))
- More recent are the books by Lohr (1999) and Scheaffer et al. (2006)
- And in 1994 StatsNZ published ‘A Guide to Good Survey Design’ (Zealand (1995))

And also

- CRAN site on R resources for Official Statistics <https://cran.r-project.org/web/views/OfficialStatistics.html>
- Online book on using R for survey analysis: https://bookdown.org/jimr1603/Intermediate_R_-_R_for_Survey_Analysis/
- This gallery of R graphs <https://r-graph-gallery.com/>

Some simple lectures on Sample Surveys

- https://r-project.ro/conference2017/presentations/Camelia_Goga_sondages.pdf
- Notes for epidemiologists: https://www.epirhandbook.com/en/new_pages/survey_analysis.html
- Notes on imputation: <https://www.r-bloggers.com/2023/01/imputation-in-r-top-3-ways-for-imputing-missing-data/>
- An online book about spatial sampling <https://dickbrus.github.io/SpatialSamplingwithR/>
- From Qualtrics <https://www.qualtrics.com/articles/strategy-research/sampling-methods/>

Data

- UC Irvine Machine Learning data repository: <http://archive.ics.uci.edu/>
- Some free microdata sets: <https://asdfree.com/>
- Kaggle data repository: <https://www.kaggle.com/>
 - Airline dataset from Kaggle: <https://www.kaggle.com/datasets/thedevastator/airlines-traffic-passenger-statistics>
 - * distributed under the Creative Commons Licence 4.0: <https://creativecommons.org/licenses/by-nc-sa/4.0/>
- Address file from LINZ: <https://data.linz.govt.nz/layer/123113-nz-addresses/>
- List of NZ schools <https://www.educationcounts.govt.nz/directories/list-of-nz-schools>

1.3 R Packages

- **survey** package (Lumley (2004)) with this handy summary of command uses <https://stats.oarc.ucla.edu/r/seminars/survey-data-analysis-with-r/>
- **srvyr** a dplyr wrapper for the **survey** package
- **gtsummary** creates outputs ready for publication (difficult to build/load)
- **sampling** package for drawing samples according to various designs
- **mice** package for imputation
- **simputation** package for imputation (difficult to build/load)
- **haven** package to import SAS, Stata and SPSS data files

Chapter 2

Sampling

In statistics the data we analyse is often a subset, i.e. a **sample** of members of a **population**.

For example we might have a sample of households whose members respond to a questionnaire, or a sample of plants from a field.

In order to select from a population we first enumerate (list) the population in some way and then make a random selection of our subset members from that list. The method by which we do so is called the **sample design**.

Our focus in statistics is often the **estimation** of population characteristics (known as **population parameters**) on the basis of sample characteristics (which are called **sample statistics**). We are always aware that the sample we drew from the population is not the only sample we could have drawn, and therefore our conclusions are subject to uncertainty, which we call **sampling error**.

This uncertainty associated with sampling is the reason we have confidence intervals on our estimates, and why we can neither **prove** nor **disprove** hypotheses.

(One exception is when we do a **census** where we select and get fully accurate data from every member of the population. Then there's no uncertainty left at all, and our sampling errors are zero.)

When generating **simulated** datasets we also generate samples. Simulation is often done as a means of testing out statistical ideas and methods, or (in the case of the bootstrap method) when conventional ways of estimating standard errors (and therefore confidence intervals) don't work.

Before we get into detail of estimation and inference, let's spend some time thinking about how sampling works.

2.1 Random numbers

In both settings (sampling and simulation) we need to start with a method of generating random numbers. We'll start by assuming we have some means of generating a random sequence of numbers uniformly distributed in the interval $[0,1]$:

$$U_i \sim \text{Uniform}(0,1) \quad \text{for } i = 1, 2, 3, \dots$$

Here 'uniformly distributed' means that all values in the range $[0,1]$ are equally likely to occur.

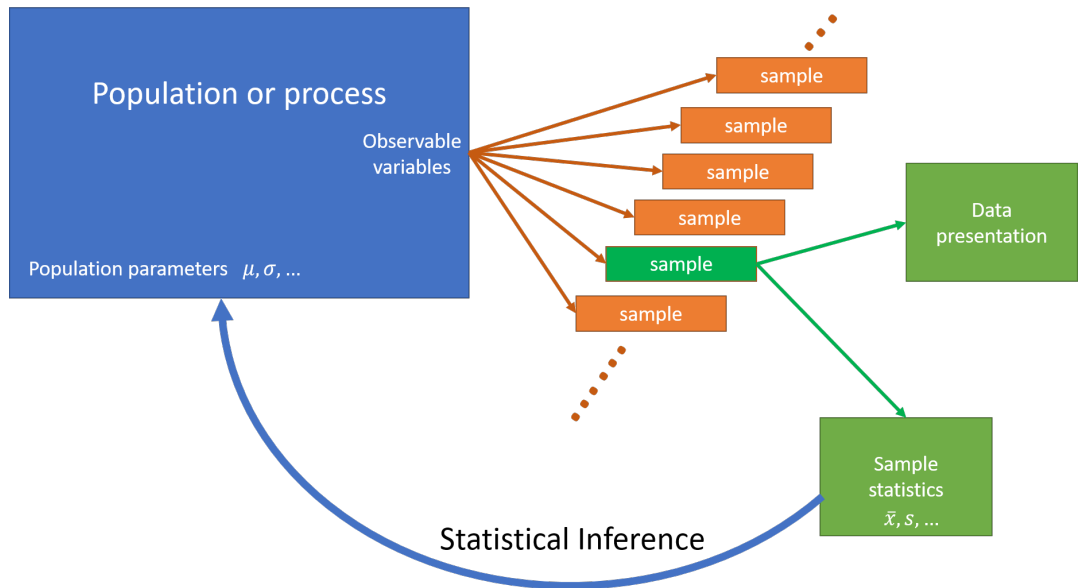


Figure 2.1: Statistical Inference

In R this is done using the `runif()` function. For example to generate 5 random draws from `Uniform(0,1)`:

```
runif(5)
```

```
## [1] 0.2875775 0.7883051 0.4089769 0.8830174 0.9404673
```

Another draw will a different set of random numbers:

```
runif(5)
```

```
## [1] 0.0455565 0.5281055 0.8924190 0.5514350 0.4566147
```

If we draw a histogram of 100 draws it looks a bit lumpy:

```
hist(runif(100), xlab="x", ylab="Frequency")
```

But a histogram of 100,000 draws is very uniform:

```
hist(runif(100000), xlab="x", ylab="Frequency")
```

Although the numbers appear to be random they are in fact generated using formulae, so if one knows the formula the sequence of random numbers generated by `runif()` is completely predictable. However the formula is complex enough that there are no easily discernible patterns in sequences of many millions of draws. These numbers are **psuedo-random**, but they are random enough for our purposes.

Underlying the random number generator is a **seed** value, which initiates the sequence. If we set the seed to a particular value, the sequence of random numbers that will be generated after doing so will always be the same.

```
set.seed(111)
```

```
runif(5)
```

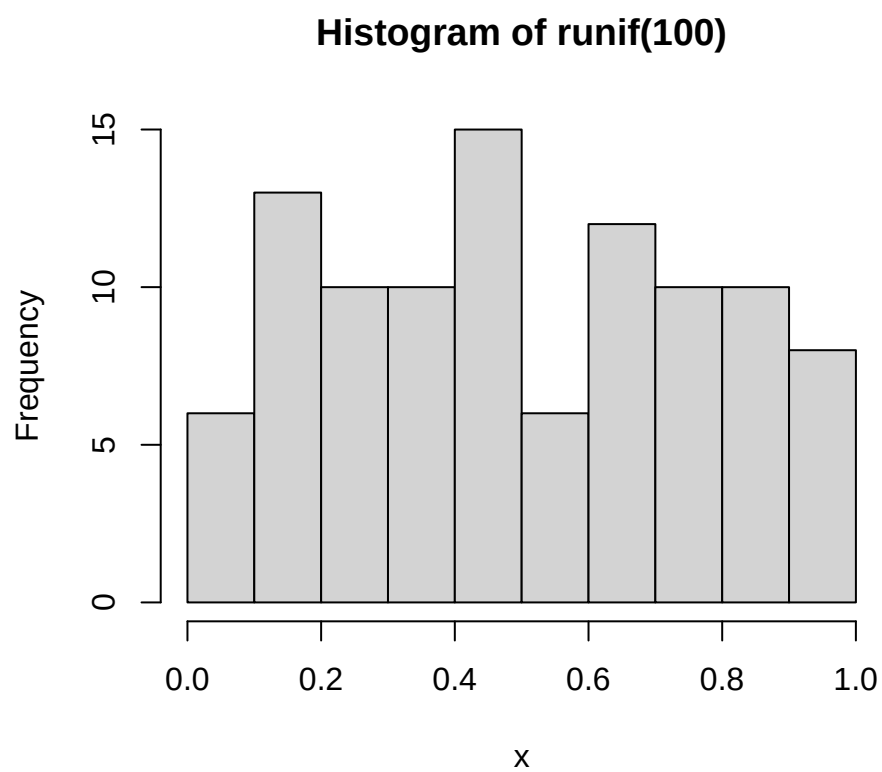


Figure 2.2: 100 draws from Uniform(0,1)

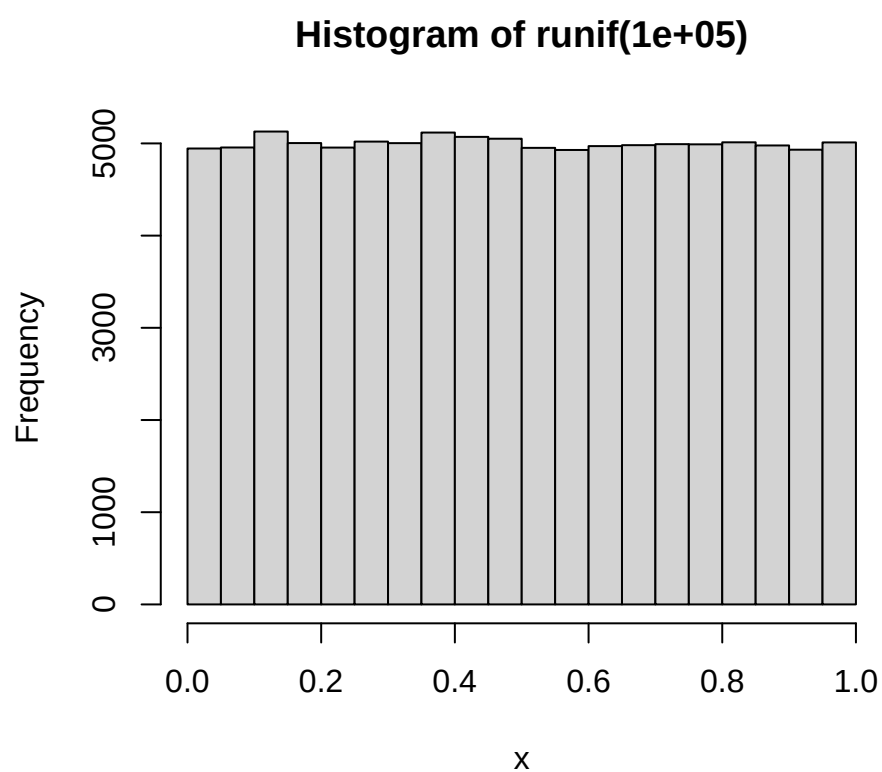


Figure 2.3: 100000 draws from Uniform(0,1)

```
## [1] 0.5929813 0.7264811 0.3704220 0.5149238 0.3776632
```

```
runif(5)
```

```
## [1] 0.41833733 0.01065785 0.53229524 0.43216062 0.09368152
```

```
runif(5)
```

```
## [1] 0.55577991 0.59022849 0.06714114 0.04754785 0.15620252
```

```
set.seed(8)
```

```
runif(5)
```

```
## [1] 0.4662952 0.2078233 0.7996580 0.6518713 0.3215092
```

```
set.seed(111)
```

```
runif(5)
```

```
## [1] 0.5929813 0.7264811 0.3704220 0.5149238 0.3776632
```

This property can be very useful when testing and comparing algorithms: we can generate an identical sequence of random numbers and check (for example) that two methods of estimation give the same results on two identical randomly generated datasets.

2.2 Sampling from categorical random variables

Draws from a uniform distribution are all we need to simulate/sample from any distribution.

Heads or tails: if the probability of flipping heads is 0.5, we can generate n random numbers, and then label any values less than 0.5 as Heads and the remainder as tails.

```
n <- 10
```

```
u <- runif(n)
```

```
u
```

```
## [1] 0.2875775 0.7883051 0.4089769 0.8830174 0.9404673 0.0455565 0.5281055
```

```
## [8] 0.8924190 0.5514350 0.4566147
```

```
y <- ifelse(u<=0.5,"Heads","Tails")
```

```
y
```

```
## [1] "Heads" "Tails" "Heads" "Tails" "Tails" "Heads" "Tails" "Tails" "Tails"
```

```
## [10] "Heads"
```

```
table(y)
```

```
## y
```

```
## Heads Tails
```

```
##      4      6
```

```
table(ifelse(runif(10000)<=0.5,"Heads","Tails"))
```

```
##
```

```
## Heads Tails
```

```
## 5059 4941
```

We can extend this idea to any set of categories.

Red, Green or Blue: Say we have a six sided die: three sides are red, two are green and one is blue. The probabilities of rolling the three colours are therefore $\frac{1}{2}$, $\frac{1}{3}$, $\frac{1}{6}$ respectively. These probabilities add to one.

The random variable Y is the colour that the die shows when we roll it.

```
pp <- c(Red=1/2, Green=1/3, Blue=1/6)
sum(pp)
```

```
## [1] 1
```

In general we might have m categories, where the probability of that Y category k is p_k : $\Pr(Y = k) = p_k$. and

$$\sum_{k=1}^m p_k = 1$$

We then compute the **cumulative probabilities** c_k , the partial sums of these probabilities:

$$c_k = \sum_{j=1}^k p_j$$

The final cumulative probability c_m is always 1 by definition.

Implicitly we also have a value $c_0 = 0$, which is the the empty sum and always equal to zero.

These cumulative probabilities c_k are 'the probability that random variable Y has a category label less than or equal to k ':

$$\Pr(Y \leq k) = c_k = \sum_{j=1}^k p_j$$

Red, Green or Blue: the cumulative probabilities are

$$\begin{aligned} c_0 &= 0.0000 \\ c_1 &= p_1 = 0.5000 \\ c_2 &= p_1 + p_2 = 0.5000 + 0.3333 = 0.8333 \\ c_3 &= p_1 + p_2 + p_3 = 0.5000 + 0.3333 + 0.1667 = 1.0000 \end{aligned}$$

```
m <- length(pp) # Number of categories
cp <- cumsum(pp)
cp
```

```
##      Red      Green      Blue
## 0.5000000 0.8333333 1.0000000
```

We then take a set of uniform random numbers $u_1, \dots, u_n$. We then want to find which of the m labels corresponds to each random number u_i .

The rule is that if u_i is less than cumulative probability c_k but greater than c_{k-1} , then we assign the label k .

There are various ways to do this, here's a version using `case_when()` from the `dplyr` package:

```
set.seed(112)
u <- runif(8)
u
```

```
## [1] 0.37667253 0.92201944 0.99187774 0.97723602 0.23629728 0.10706678 0.03915213
## [8] 0.72802690
```



```
y <- case_when(u <= cp[1] ~ "Red",
               u <= cp[2] ~ "Green",
               .default="Blue") # here .default means "otherwise"
data.frame(round(u,5),y)
```

```
##   round.u..5.    y
## 1    0.37667   Red
## 2    0.92202  Blue
## 3    0.99188  Blue
## 4    0.97724  Blue
## 5    0.23630   Red
## 6    0.10707   Red
## 7    0.03915   Red
## 8    0.72803  Green
```

```
table(y)
```

```
## y
## Blue Green  Red
##    3     1     4
```

```
table(y)/length(y)
```

```
## y
## Blue Green  Red
## 0.375 0.125 0.500
```

Note that the categories in the table haven't come out in the Red-Green-Blue order that we want - instead they're in the default alphabetical order.

We can fix this by converting `y` to a `factor` object:

```
y <- factor(y, levels=names(pp))
table(y)
```

```
## y
## Red Green  Blue
##    4     1     3
```

Here's what is really going on: we if we plot the cumulative probabilities as a staircase – a **cumulative distribution function (CDF)** then we draw u_i from a Uniform(0,1) distribution, and then find where a horizontal line drawn at the level of u_i intersects the CDF: and that is the value of the categorical variable y_i .

The larger the probability p_k the bigger the jump in the staircase, and the larger the probability that category k will be selected.

```
set.seed(112)
n <- 10000
u <- runif(n)
y <- case_when(u <= cp[1] ~ "Red",
               u <= cp[2] ~ "Green",
               .default="Blue")
y <- factor(y, levels=names(pp))
table(y)
```

```
## y
```

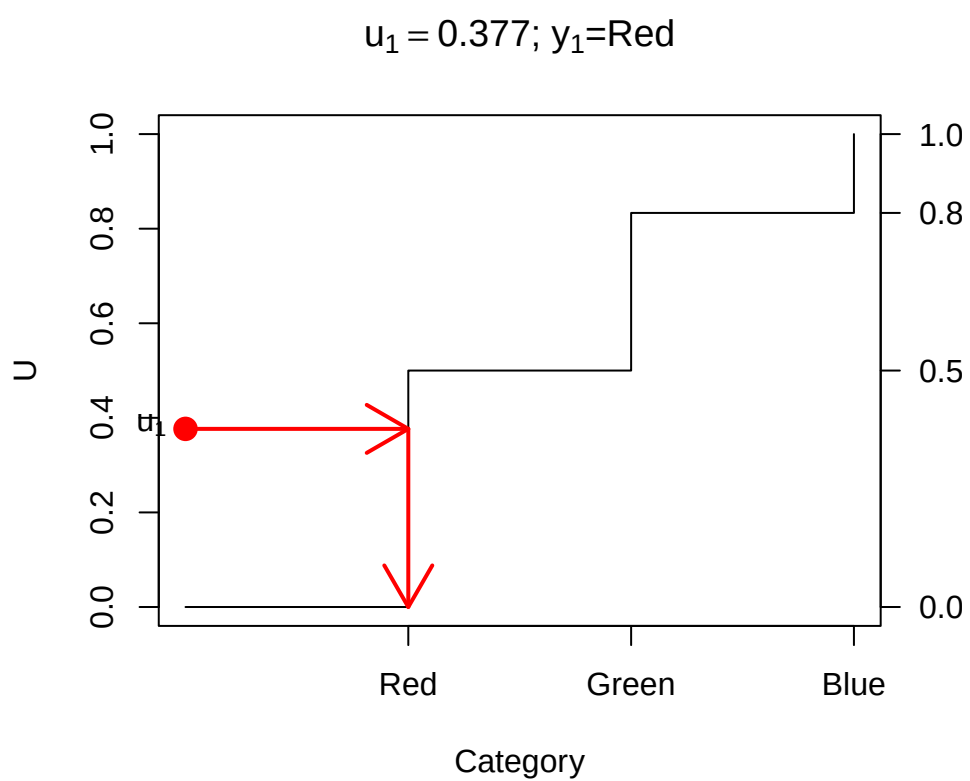


Figure 2.4: Drawing a random category

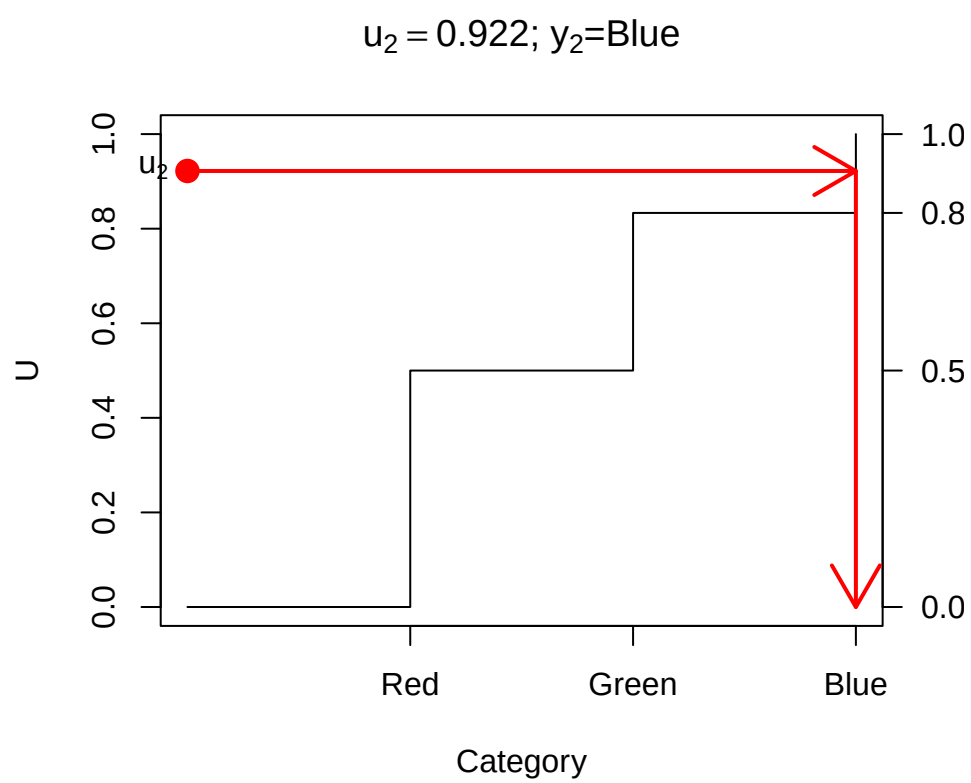


Figure 2.5: Drawing a random category

```
##   Red Green  Blue
##  4976  3336  1688
```

The `sample()` function implements this conveniently: select from the integers 1 to `m`

```
n <- 10000
y <- sample(m, size=n, prob=pp, replace=TRUE)
table(y)
```

```
## y
##   1    2    3
## 5014 3350 1636
```

(need `replace=TRUE` here, see below).

... or state the vector `1:m` explicitly

```
y <- sample(1:m, size=n, prob=pp, replace=TRUE)
table(y)
```

```
## y
##   1    2    3
## 5010 3371 1619
```

... or directly from the vector of labels ...

```
y <- sample(c("Red", "Green", "Blue"), size=n, prob=pp, replace=TRUE)
y <- factor(y, levels=names(pp))
table(y)
```

```
## y
##   Red Green  Blue
##  4980  3325  1695
```

... get the labels from `pp` ...

```
y <- sample(names(pp), size=n, prob=pp, replace=TRUE)
y <- factor(y, levels=names(pp))
table(y)
```

```
## y
##   Red Green  Blue
##  4994  3389  1617
```

2.3 Sampling from discrete numerical random variables

The ideas above carry straight over to discrete numerical random variables.

The Binomial distribution $Y \sim \text{Bin}(m, p)$ is the number of successes in m trials where the probability of success on each trial is the same p , and all the trials are independent.

Y takes values 0 to m , and those probabilities can be calculated by

$$P(Y = k) = \binom{m}{k} p^k (1-p)^{m-k} = \frac{m!}{k!(m-k)!} p^k (1-p)^{m-k}$$

R provides these probabilities in the function `dbinom()`

e.g. $m = 10$ rolls of a six-sided die, and Y is the number of times we get a 1, so $p = 1/6$

```

m <- 10
p <- 1/6
pp <- dbinom(0:m, m, p)
pp

## [1] 1.615056e-01 3.230112e-01 2.907100e-01 1.550454e-01 5.426588e-02
## [6] 1.302381e-02 2.170635e-03 2.480726e-04 1.860544e-05 8.269086e-07
## [11] 1.653817e-08

barplot(pp, names=0:m,
        xlab="y", ylab="Pr(Y=y)",
        main=bquote("Bin("*. (m)*", ".*. (sprintf(" %.4f", p))*")"))

```

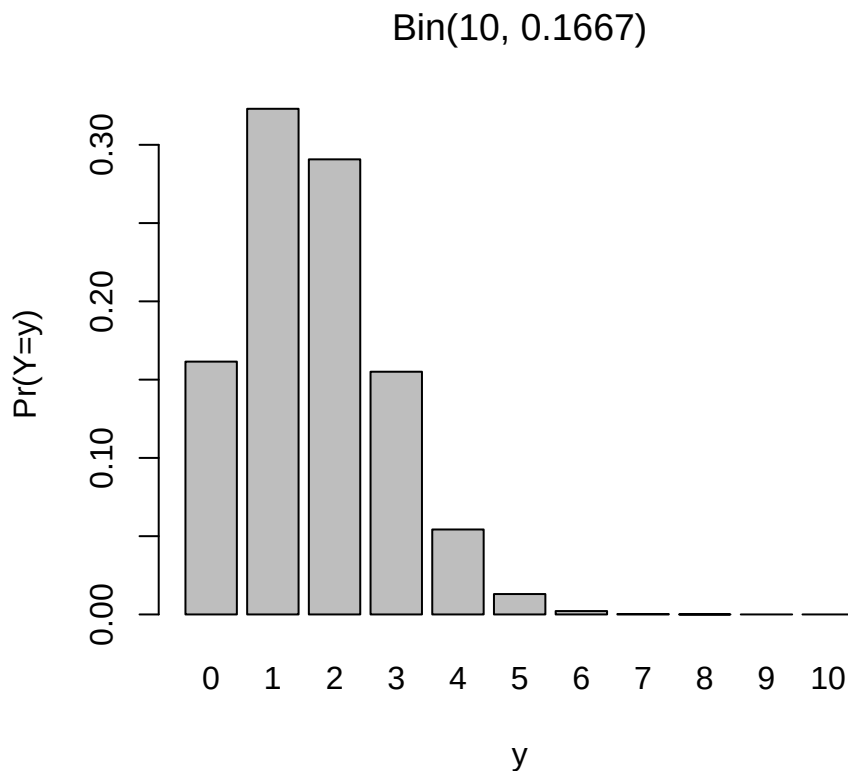


Figure 2.6: Binomial probabilities

The CDF is given by the function `pbinom()`

```

cp <- pbinom(0:m, m, p)

plot(0:m, cp, type="s",
     xlab="y", ylab="Pr(Y<=y)",
     main=bquote("Bin("*. (m)*", ".*. (sprintf(" %.4f", p))*")"))

```

Drawing a single value of Y proceeds the same way as for a categorical variable:

So we can draw multiple values of Y from $\text{Bin}(10, \frac{1}{6})$ using the `sample()` function:

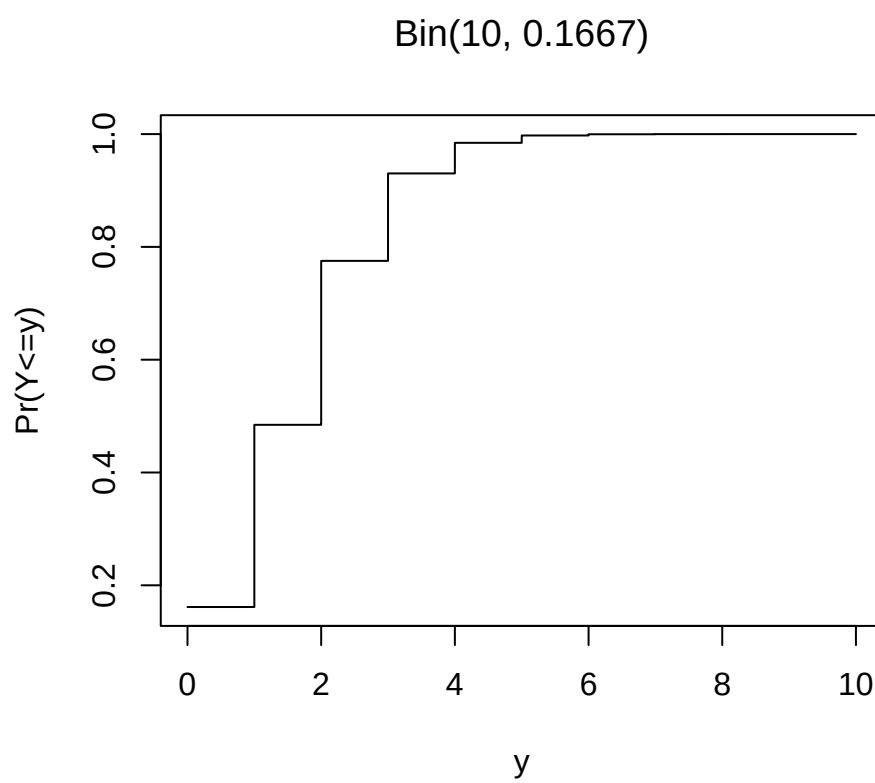


Figure 2.7: Binomial CDF

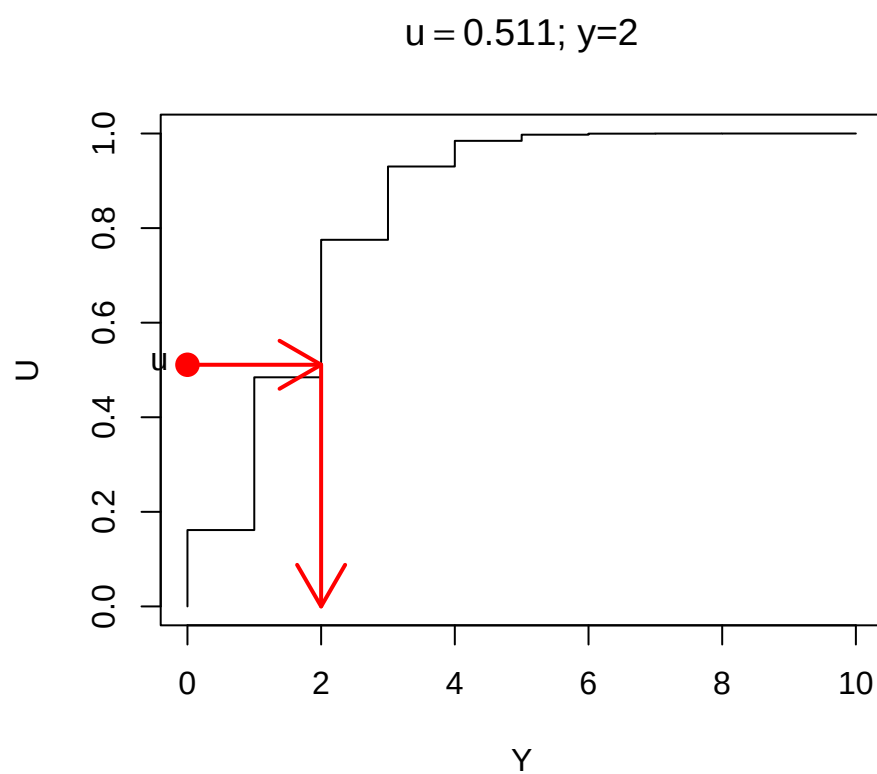


Figure 2.8: Drawing from a Binomial(10,1/6) distribution

```
n <- 10000
pp <- dbinom(0:m, m, p)
y <- sample(0:m, n, prob=pp, replace=TRUE)
barplot(table(y))
```

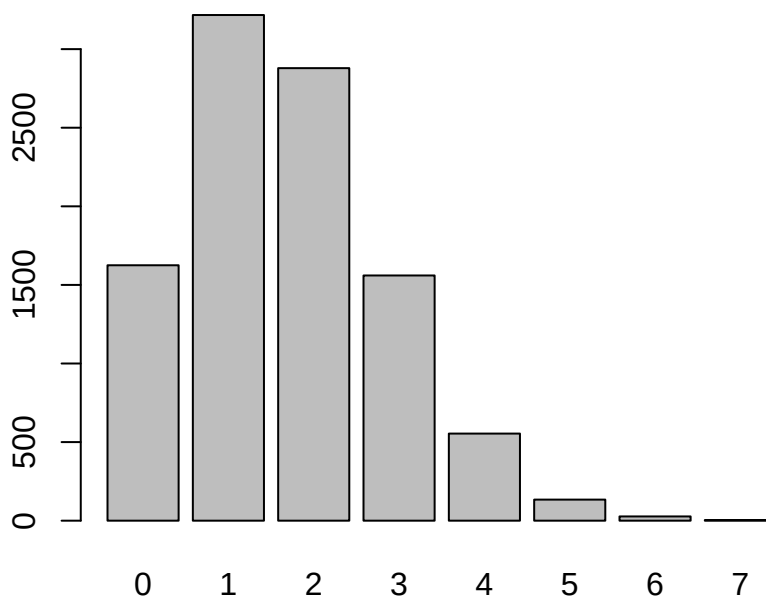


Figure 2.9: 10000 draws from a Binomial(10,1/6)

Of course R provides a specific function for doing the same thing: `rbinom()`

```
n <- 10000
y <- rbinom(n, m, p)
barplot(table(y))
```

But that function is just doing what we've described above using `sample()`.

A particular common example is where we have m objects and want to select one of them at random, with each object having equal probability of being selected: i.e. each has probability $1/m$.

2.4 Sampling from continuous random variables

Continuous random variables are simulated using the same principle.

The cumulative distribution function (CDF) of a continuous random variable Y is defined to

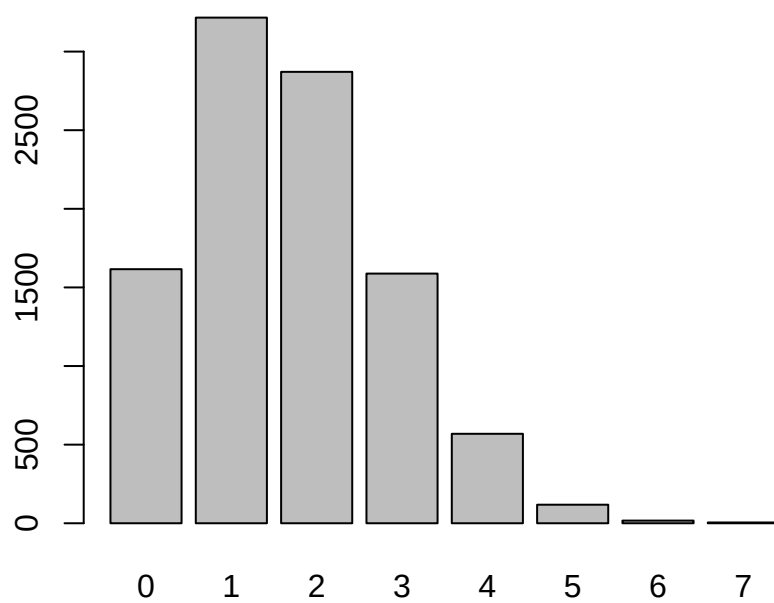


Figure 2.10: 10000 draws from a Binomial(10,1/6)

be

$$F(y) = \Pr(Y \leq y) = \int_{-\infty}^y f(y') \, dy'$$

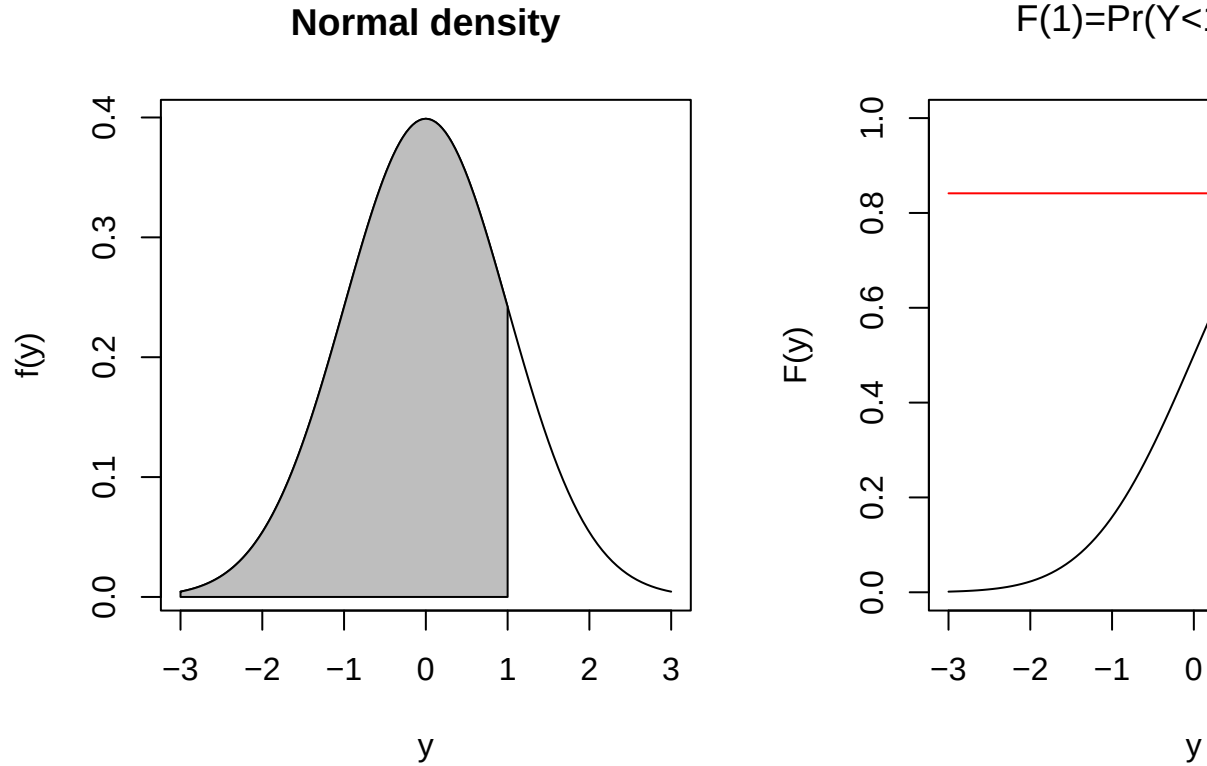


Figure 2.11: Normal distribution density and CDF

We can use $F(y)$ to draw a random value from the distribution of Y by first drawing a random value U from a $\text{Uniform}(0,1)$ distribution.

We then solve the equation

$$U = F(y)$$

for y : i.e. find the value of y corresponding to a CDF value of U .

For many standard distributions (including the Normal and Gamma distributions) there is no analytic formula which gives the solution for y given u : but there are numerical approximations which can be used in those settings.

One simple case which does have an analytic solution is the Exponential distribution $Y \sim \text{Exp}(\lambda)$. This is a one parameter distribution where Y is non-negative ($Y \geq 0$), and can be used to model cases such the length of time it takes for an event to occur (e.g. the length of time for a radioactive nucleus to decay). The single parameter λ is the **rate** (e.g. number of radioactive decay events per unit time).

The exponential distribution has probability density function

$$f(y) = \lambda e^{-\lambda y} \quad \text{where } y \geq 0$$

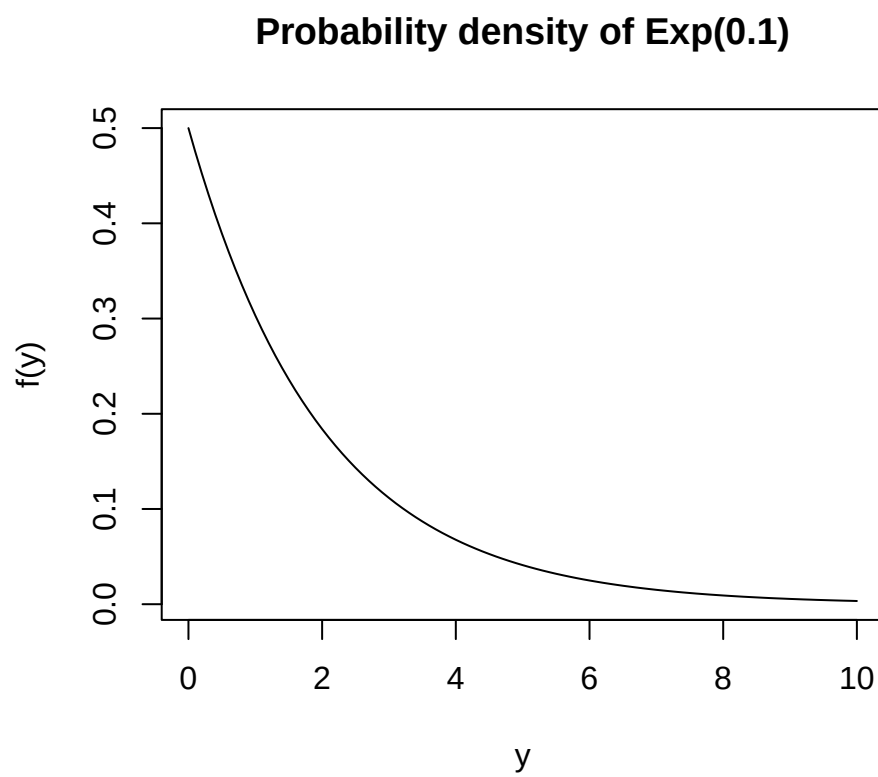


Figure 2.12: Exponential probability distribution

and has mean $E[Y] = 1/\lambda$.

The CDF is the integral of the probability density function

$$\Pr(Y \leq y) = F(y) = \int_0^y f(y) dy$$

which for the Exponential distribution is

$$F(y) = \int_0^y \lambda e^{-\lambda t} dt = 1 - e^{-\lambda y}$$

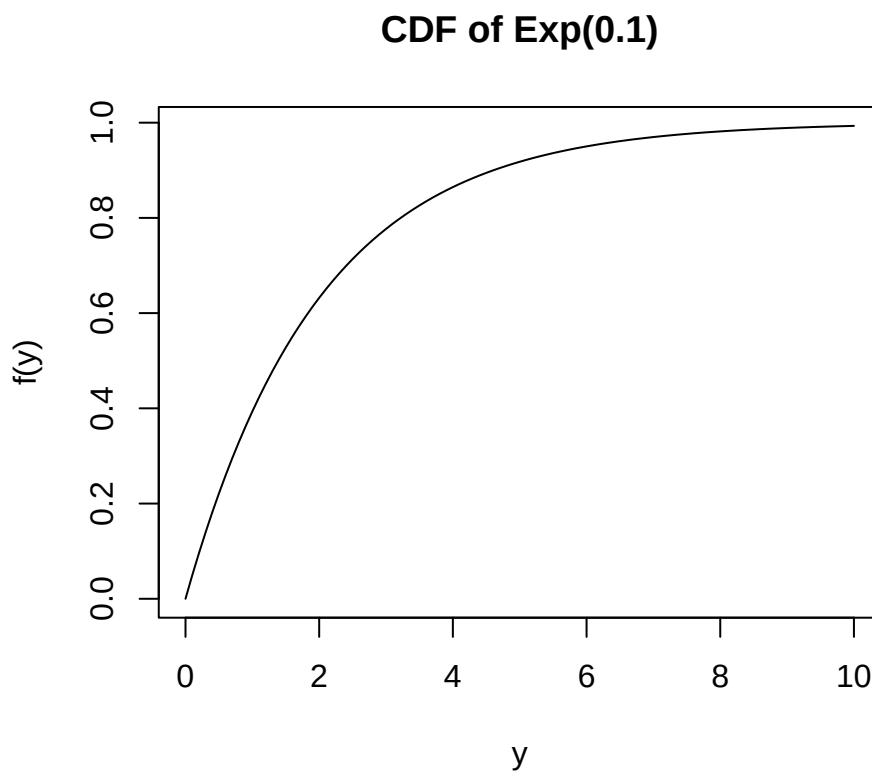


Figure 2.13: Exponential cumulative probability distribution

Solving

$$u = F(y) = 1 - e^{-\lambda y}$$

for y leads to

$$y = -\frac{1}{\lambda} \log(1 - u)$$

Using this function we can easily draw directly from an Exponential distribution.

```
n <- 10000
u <- runif(n)
lambda <- 0.5
y <- -(1/lambda)*log(1-u)
mean(y)
```

```
## [1] 1.990456
```

```
hist(y, main="Histogram of draws from Exp(0.5)")
```

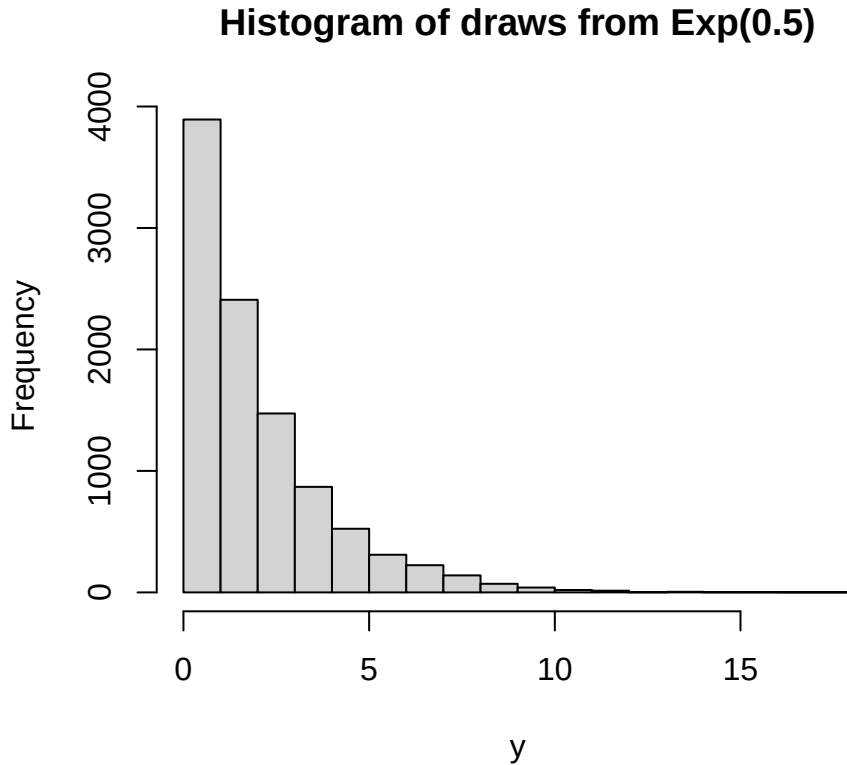


Figure 2.14: 10000 draws from an Exponential distribution

2.5 Sampling from a finite collection

The simulations discussed so far all draw from **infinite** populations: we can set the sample size n to any value. However it often occurs that we have only a finite population - e.g. when drawing a random sample of students from a class, or households from a suburb for interview. Then the population size N is finite.

In the general case we have a set of N **units** (i.e. students or households) in a population of finite size, and we want to draw a sample of n of these units.

We usually start by listing or **enumerating** the units. This list is called a **frame** or a **sample frame**.

Consider the case where we have a small street containing $N = 10$ houses, numbered 1-10. We want to select 5 of these houses with equal probability. There are a number of ways we could do this.

2.5.1 Sampling without replacement

If the houses have equal probability then we could proceed as follows:

- We draw a random number from 1-10, e.g. house number 2, and mark it as selected
- We draw another random number from 1-10, e.g. house number 5, and mark it as selected
- We repeat this process, but if we choose a house that is already selected, we discard that selection;
- The process continues until we have a set of 5 distinct houses.

At the start we take our frame (i.e. list) of houses, and initially mark them all as unselected. Then at each step where we successfully select a previously unselected house we mark that house as selected.

```
bigN <- 10 # population size
frame <- data.frame(house=1:bigN, selected=FALSE)
n <- 5 # sample size
nselected <- 0
while(nselected < n) {
  i <- sample(bigN, 1) # sample a single number at random from 1:bigN with equal prob
  if(!frame$selected[i]) {
    frame$selected[i] <- TRUE
    nselected <- nselected + 1
  }
}
frame$house[frame$selected]
```

```
## [1] 1 6 7 8 10
```

It's not guaranteed how many steps this process is going to take, since the number of times we select a house that has already been selected is random. If the sample size n is small and the population size N is large then it will take n steps, or possibly just a few more. If n is close to N and N is large, it could take a very very long time.

A simpler way to solve this is to assign each house a distinct random number between 0 and 1, and then sort the frame in order of the random numbers.

```
bigN <- 10 # population size
frame <- data.frame(house=1:bigN, selected=FALSE)
n <- 5 # sample size
frame$u <- runif(bigN)
frame <- frame[order(frame$u),]
frame$selected[1:n] <- TRUE
frame
```

```
##   house selected      u
## 9      9      TRUE 0.0812427
## 4      4      TRUE 0.3739315
## 2      2      TRUE 0.4137952
## 6      6      TRUE 0.4341604
## 10     10      TRUE 0.4909159
## 3      3     FALSE 0.5850114
## 8      8     FALSE 0.8708720
## 5      5     FALSE 0.8837804
## 1      1     FALSE 0.9278206
## 7      7     FALSE 0.9475266
```

```
frame <- frame[order(frame$house),]
frame$house[frame$selected]
```

```
## [1] 2 4 6 9 10
```

Of course, the simplest way of all is just to use the `sample()` function

```
bigN <- 10 # population size
n <- 5 # sample size
sample(bigN, n)
```

```
## [1] 10 4 6 3 2
```

It's very often that we do have a frame, a list of objects with names, and we want to select from this at random.

```
frame <- data.frame(Name=c("Mark","Chloe","Sam","Caitlin","Harry","Nina"),
                    Age=c(20,19,18,19,19,21),
                    Major=c("STAT","STAT","DATA","COMP","MATH","STAT"))
frame
```

```
##      Name Age Major
## 1    Mark  20  STAT
## 2   Chloe  19  STAT
## 3     Sam  18  DATA
## 4 Caitlin  19  COMP
## 5   Harry  19  MATH
## 6    Nina  21  STAT
```

We can select, say, 3 names

```
n <- 3
samplenames <- sample(frame$Name, n)
samplenames
```

```
## [1] "Nina" "Sam" "Mark"
```

and then subset the frame

```
samplemembers <- frame[frame$Name%in%samplenames,]
samplemembers
```

```
##      Name Age Major
## 1 Mark  20  STAT
## 3 Sam  18  DATA
## 6 Nina  21  STAT
```

If you like dplyr

```
samplemembers <- frame %>%
  filter(Name%in%samplenames)
samplemembers
```

```
##      Name Age Major
## 1 Mark  20  STAT
## 2 Sam  18  DATA
## 3 Nina  21  STAT
```

Note we can find out where in the frame particular individuals are located

```
match(c('Mark','Harry'),frame$Name)
```

```
## [1] 1 5
```

If we're sampling without replacement, note (of course) that the sample size can be no bigger than the population size $n \leq N$.

2.5.2 Sampling with replacement

When selecting a sample of 5 houses it makes sense not to allow the possibility of selecting the same house twice. So when we've selected a house we remove it from the list of houses available to select. When using `sample()` this means that we set `replace=FALSE`, meaning after we've selected a member of the population we don't put it back again.

But there are occasions where we select **with replacement** is the sensible thing to do. This means there is a chance where a unit can be selected more than once, and this occurs in situations where we want to assign different probabilities of selection to different units of the population.

Starting with the the case of equal probabilities, we repeatedly draw units from the population under identical circumstances: on every draw each unit is available for selection, and we count how many times each unit is selected.

```
set.seed(111)
bigN <- 10
n <- 5
replicate(n, sample(bigN, 1))
```

```
## [1] 4 3 9 5 3
```

In this sample we've drawn Unit 3 twice.

If n is small and $bigN$ is large the probability of repeats is small. But if a repeat happens we of course do not make our selected household repeat the questionnaire, instead we count them twice (or as many times as they are selected) in our analysis.

Imagine our sample of 10 houses aren't actually just houses, they're farms, and some of them are very large and some are very small. We want to give the large farms a high chance of selection.

```
set.seed(122)
bigN <- 10
n <- 5
frame <- data.frame(farm=1:10, ncows=c(1000,5000,2000,500,500,
                                         5000,500,5000,30000,100))
frame
```

```
##      farm ncows
## 1      1  1000
## 2      2  5000
## 3      3  2000
## 4      4   500
## 5      5   500
## 6      6  5000
## 7      7   500
## 8      8  5000
## 9      9 30000
## 10     10   100
```

```
sample(frame$farm, n, prob=frame$ncows, replace=TRUE)
```

```
## [1] 2 2 9 9 9
```


Unsurprisingly, the very large farm number 9 has been selected three times here.

It **is** possible to have unequal probability sampling **without** replacement, estimation of standard errors is complicated in such designs, so we won't be using such designs.

For now, just note that we switch to with replacement designs when we're sampling from sets of units where we want to have units with different probabilities of selection.

Chapter 3

Sample Surveys

A sample survey is an exercise in which **data** are collected from a **sample** (a subset) or a **population**. The data collected are used to create **estimates** of the characteristics or **parameters** of the population.

Some examples:

- A sample of voters are telephoned to ask their voting intentions, with the aim of predicting the outcome of the election;
- A sample of sites in a river catchment is tested for the presence of algae to determine the spread of the algae throughout the entire catchment;
- A sample of households is selected, and the residents asked about their employment status, in order to estimate the national unemployment rate.

A **census** is a special case of a sample survey in which every member of the population is surveyed.

3.1 The Survey Process

The different steps in the survey process are shown in Figure 3.1.

- Every survey has a set of **objectives** – the particular populations which are of interest;
- These parameters are properties of the **target population**: the population about which estimates are to be made;
- Not all members of the target population are able to be identified: the ones that can be surveyed make up the **survey population**;
- The **sample frame** is a listing of all members of the survey population;
- The **sample design** is the method of selecting the sample from the frame. The **sample size** is usually decided as a compromise between the required **accuracy of estimates** and the survey **costs** and other constraints (e.g. the time available for the survey);
- **survey instrument** is the means of data collection. It is usually a paper or electronic questionnaire, completed by the respondent or the interviewer/observer. The instrument aims to measure the properties of the sample members which mean that the desired population characteristic can be estimated. The instrument must be **valid** (it must actually measure what it intends to measure) and **reliable** (repeated measurements of the same sample member under identical circumstances should always yield similar results);
- The sample members are **contacted** and **recruited** into the sample. Not all selected members will be able to be contacted (even after strenuous efforts), and some will not

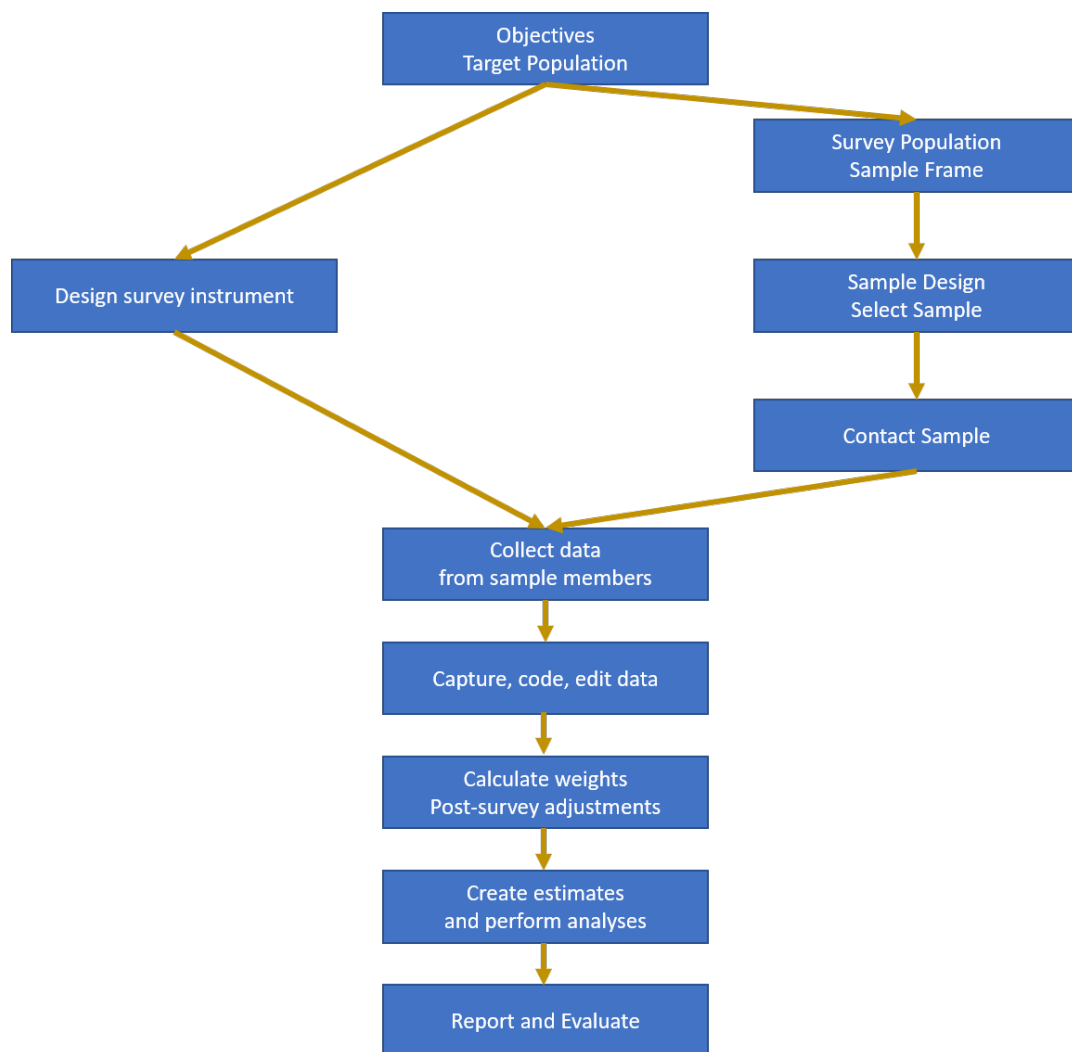


Figure 3.1: The Survey Process

- respond even if contacted;
- The data are collected from the sample members in some **mode** (e.g. face-to-face, telephone, web, observational, ...);
- The data collected are **captured** (stored on a computer); **coded** (converted into standard classification systems); **edited** (checks for data consistency); and **stored** in a final dataset;
- The data may be **adjusted**, e.g. imputation or weight adjustments for nonresponse may be done;
- Estimates** of the parameters of interest are constructed, and other **analysis** of the data is carried out (e.g. regression estimation, comparison with other data etc.);
- The results are summarised in a **report**;
- The original data may be **archived** in some appropriate form, or **destroyed**;
- A post-survey **evaluation** may be made to determine how well the survey met its goals.

For example: The Household Labour Force Survey (HLFS).

- One of the main **objectives** of the HLFS is to estimate the unemployment rate every quarter;
- The **target population** is the working age population of New Zealand;
- The **survey population** is the civilian, non-institutionalised usually resident population of adults aged 15+ living in permanent, private dwellings on the main islands of New Zealand (North Island, South Island and Waiheke Island);
- The **sample frame** is a list of dwellings created at the most recent census and regularly updated to reflect changes;
- The **sample design** is a stratified cluster design. Within each regional government area a sample of small areas (PSUs) is selected. A sample of households is selected from within each PSU. Every adult from the selected households is surveyed. 15000 households and 30000 adults are surveyed every three months, in order to create unemployment estimates accurate to within 0.5%.
- The **survey instrument** is an electronic questionnaire.
- The interviewers contact the households in person or by telephone, making up to 10 call backs to ensure contact is made. **Proxy responses** are permitted (i.e. one household member can respond on behalf of another). The interview takes place by computer assisted personal or telephone interviewing (CAPI or CATI).
- The results are **post-stratified** to match the current population estimates in each local government area;
- Estimates** of the unemployment rate are published within about 6 weeks of the end of the quarter.

3.2 Populations and Frames

The target population defines the **scope** of the survey: the set of units which make up the population in which we are interested.

The sample frame is a method of gaining access to the members of the target population. It is an actual or conceptual list. However it may not be perfect (Figure 3.2).

- Not all members of the target population will appear on the sample frame: the missing units are **out of coverage**;
- Not all units listed on the sample frame are members of the target population: the extra units are **out of scope**.
- The survey population are those units which are both in the target population (in scope) and on the frame (in coverage).

Our aim is to have a sample frame which covers most if not all of the target population, and which has only very few out of scope units.

The reason for this is very important: we can make estimates about units in a population **only** if they are included in the sample frame. This inference is possible **whether or not we select them**. If there is a subpopulation of units that is not on our frame (i.e. they are out of coverage) then we can never select them, we can never learn anything about them, and therefore we cannot make inferences about them.

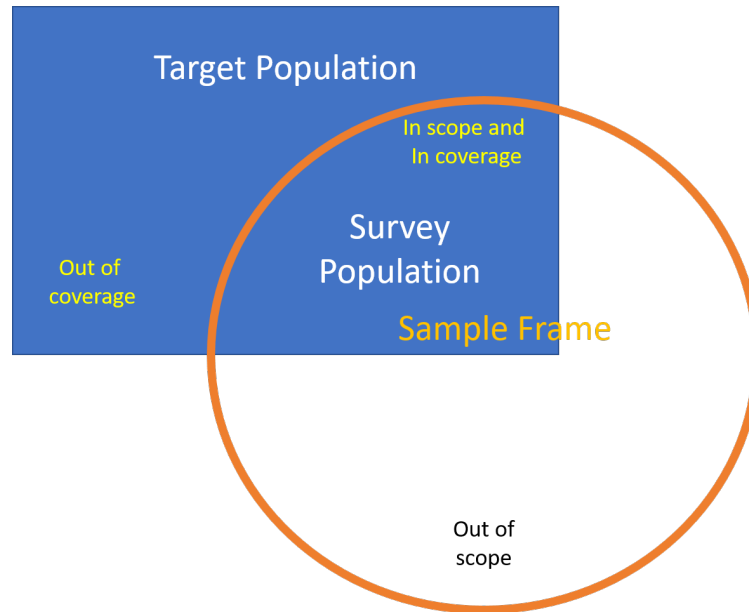


Figure 3.2: Scope and Coverage

3.2.1 What is a sampling frame?

The sampling frame is a key element in the survey process which provides the basis from which a *random* (probability based) sample can be selected. The purpose of having a sampling frame is to enable each member of the target population a nonzero probability of being selected to participate in the survey. The *quality* of the data, from a survey which has as its basis a comprehensive frame, can be *measured and controlled*.

The term sampling frame is sometimes misunderstood, often being thought of as a list containing each member of the population. In fact that is more a description of an ‘ideal’ sampling frame. The best way to show what a sampling frame can be in practice is to examine some examples:

- **List Frames.** Good up-to-date lists make ideal frames. However lists of units (electoral rolls, telephone listings etc.) are generally formed for purposes other than for use as sampling frames. For this reason they are often not close to ‘ideal’ sampling frames. Some discussion of list frame problems comes later.
- **Multi-stage Sampling Frames.** These frames consist of a set of lists which are arranged in a hierarchy of stages. For example a typical household survey frame will be organised like so: The first list is a set of non-overlapping geographic areas from which a sample of areas is taken. For example we may list all the local regional councils and take a sample of them. Within these areas we might then list all the households

and take samples from the household lists. From each household we would then list the residents and interview selections of them. Note that we do not have to list the entire population. The only comprehensive list needed is the list of the areas which will be quite small. By giving each area a chance of being selected we ensure each person living in each area has a chance of selection.

- **Continuous Sampling Frames.** Consider the population of people entering a country from international flights. Suppose we choose a random number between one and ten ($= k$) and then interview the k th, $(k + 10)$ th, $(k + 20)$ th ... persons who file through customs. We will have a sample where every person has had a 1 in 10 chance of being selected. But what is the frame? There is no list. We call such a situation a ‘conceptual list’, where the people filing into the country effectively formed a long queue of people which we have treated like a listing of the population, although a listing of names was never constructed. \end{description}

One can find many different definitions of sampling frames. The following is a simplified definition that should serve our purposes:

A *sampling frame* is part of the system of rules and procedures used when determining who is to be surveyed and which ensure that each member of the population get some measurable (non-zero) chance of participating in the survey.

The examples above show that we need to think beyond the concept of a list of units, whose construction is a separate exercise from the other survey stages.

3.2.2 What is a *good* sampling frame?

We will often have several frames available and will have to decide between different options. To help us with this decision we need to be a little more specific about what makes a good sampling frame. The first thing we should consider is the effective coverage of the sampling frame. This can be determined by testing it for the following properties:

- **Each unit should be counted at least once.** If there are missing units (out of coverage) then the survey results may be biased if the excluded units tend to be different to the units which are included.
- **Each unit should be counted only once.** The problem is that we usually can not tell how many times a unit is duplicated. This means we can not tell how many chances a unit has of being sampled. This can cause similar bias if the units which are duplicated are different to the other units. The results will be biased towards the duplicated sub-group of the population.
- **Each unit should be distinguishable from the other units on the frame.** If we select a unit from the frame (say Jane Smith) but then can not tell who this refers to, when there are many Jane Smiths in the population, then the frame has not done its job.
- **Should provide up-to-date information about the units.** For example if it claims to provide Jane Smith’s address it should be the current address.
- **No units should be there if they are not meant to be.** Extra units (out of scope) are not a serious problem unless we fall into the elementary error of replacing them whenever we come across them in the sample. This confuses selection probabilities. The correct procedure is to drop any ineligible unit found in the sample and not replace them.
- **Should allow direct access to each unit.** Ideally the frame will ultimately supply us with a list of the units we are interested in analyzing. Sometimes a frame will only

give samples of groups of units e.g. a list of addresses, each of which may contain many people. Further work is needed before a sample of people is realized.

3.3 Approaches to sampling

We're talking here about random sampling, or **probability sampling**. For the purposes of statistical inference such sampling is the only defensible method - it's the only one that leads us to properly constructed confidence intervals and hypothesis tests. Other approaches do exist though, and even though you'll see them used they all have significant defects.

- **Probability sampling** is the gold standard: every member of the population has a known, non-zero chance of selection;
- **Purposive sampling**: the surveyor selects members of the population that they consider to be representative - dangerously biased towards the views and prejudices of the surveyor;
- **Haphazard sampling**: the surveyor uses any members of the population that they come across. This is what happens on the street corner when you're asked questions by a surveyor;
- **Quota sampling**: resembles stratified sample (see later): the surveyor fills quotas of respondents to balance numbers of respondents in categories (e.g. by age/gender/ethnicity), but does not make an accounting for non-response (see later);
- **Snowball sampling**: used in rare populations that are highly connected (e.g. refugees from a particular country): existing sample members recruit new sample members using their contacts;
- **Self-selected sampling**: this is the **worst** of all: the respondents themselves decide whether to be selected: they see an advertisement on a webpage or phone in with their views. Only profiles motivated population members - the silent majority are never measured.

3.4 Example datasets

We'll use and re-use a number of standard datasets.

3.4.1 API

This is a dataset on Student performance in California schools. The dataset is distributed with the `survey` library in R, and there are a number of unit record datasets which are drawn from a population dataset `apipop` under various sample designs. The population units are the schools, and a number of characteristics are available. Full documentation can be found in the `survey` package. The variables we use mostly are:

- `stype` - School type (Elementary/Middle/High School)
- `snum` - School number
- `dnum` - District number
- `api99` - Academic Performance Indicator for the school in the year 1999
- `api00` - Academic Performance Indicator for the school in the year 2000
- `sch.wide` - Did the school meet its school-wide growth target?
- `awards` - Is the school eligible for an awards program
- `meals` - Percentage of students eligible for subsidised meals
- `ell` - Percentage of students that are English Language Learners
- `grad.sch` - Percentage of students with parents with a postgraduate education
- `enroll` - Number of students enrolled
- `api.stu` - Number of students tested in the API

3.4.2 From the dplyr package

- `starwars` - a list of characteristics of 87 characters from the Star Wars movie series;

Chapter 4

Statistical Inference

4.1 Inference in infinite populations

Let's briefly review a few familiar results from earlier statistics courses.

Most of what we've done in earlier courses has assumed that we have a **simple random sample** from an **infinite** population.

We'll use data from the `starwars` dataset (in the `dplyr` package). There are data on 87 characters, including their heights in centimetres.

Six heights are missing, but let's just guess them so we have no missing data (this process of inventing data has a respectable name: **'imputation'**, and we'll come back to it later).

```
library(dplyr)
data(starwars)
sum(is.na(starwars$height))

## [1] 6

starwars <- starwars %>%
  mutate(height=case_when(name=="Arvel Crynyd" ~ 172, # same as Luke
                           name=="Finn" ~ 172, # same as Luke
                           name=="Rey" ~ 150, # same as Leia
                           name=="Poe Dameron" ~ 172, # same as Luke
                           name=="BB8" ~ 96, # same as R2D2
                           name=="Captain Phasma" ~ 167, # same as C3PO
                           .default=height))
```

The mean height $\bar{Y}$ of characters in the Star Wars universe is a **population parameter**. We can **estimate** it from a sample of characters.

Note: we're using capital letters for **population parameters** ($\bar{Y}$ is the population mean), and lower case letters for **sample statistics** ($\bar{y}$ is the sample mean).

It's also common to see Greek letters for **population parameters** (e.g. μ for the population mean), upper case latin letters for random variables, and lower case latin letters for sample statistics.

For the purposes of this exercise let's imagine there are only $N = 87$ individuals in the entire Star Wars universe. Let's draw a random sample of $n = 20$ from this population of $N = 87$:

name	height	mass	hair_color	homeworld
Tarfful	234	136	brown	Kashyyyk
Jek Tono Porkins	180	110	brown	Bestine IV
Bossk	190	113	none	Trandosha
Lando Calrissian	177	79	black	Socorro
IG-88	200	140	none	NA
Kit Fisto	196	87	none	Glee Anselm
Biggs Darklighter	183	84	black	Tatooine
Obi-Wan Kenobi	182	77	auburn, white	Stewjon
Quarsh Panaka	183	NA	black	Naboo
Mace Windu	188	84	none	Haruun Kal
Ratts Tyerel	79	15	none	Aleen Minor
Beru Whitesun Lars	165	75	brown	Tatooine
Chewbacca	228	112	brown	Kashyyyk
Jabba Desilijic Tiure	175	1358	NA	Nal Hutta
Rugor Nass	206	NA	none	Naboo
Sly Moore	178	48	none	Umbara
Dormé	165	NA	brown	Naboo
Leia Organa	150	49	brown	Alderaan
Finn	172	NA	black	NA
Darth Maul	175	80	none	Dathomir

```
set.seed(222)
bigN <- nrow(starwars)
n <- 20
samplemembers <- starwars[sample(bigN, n, replace=FALSE),]
samplemembers %>%
  dplyr::select(name,height,mass,hair_color,homeworld) %>%
  kbl() %>%
  kable_styling()
```

Compute the sample mean $\bar{y}$ and standard deviation s_y :

```
ybar <- mean(samplemembers$height)
ybar
```

```
## [1] 180.3
```

```
sd_y <- sd(samplemembers$height)
sd_y
```

```
## [1] 31.13147
```

We know how to make a point estimate $\widehat{\bar{Y}}$ for the population mean $\bar{Y}$ (remember that a hat $\widehat{\cdot}$ over a symbol indicates an estimate of that parameter): the population mean is estimated by the sample mean $\bar{y}$:

$$\widehat{\bar{Y}} = \bar{y} = 180.3$$

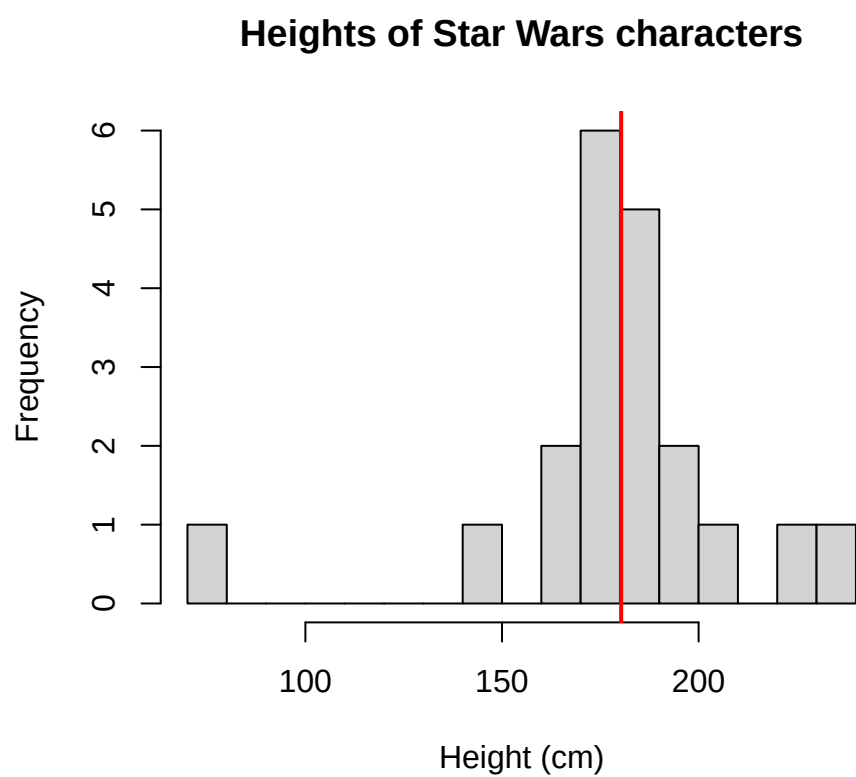


Figure 4.1: Histogram of heights of a sample of Star Wars characters (vertical line at the sample mean)

and a 95% confidence interval is given by

$$\widehat{Y} \pm \text{MOE}[\widehat{Y}] = \widehat{Y} \pm T_{n-1}^* \times \text{SE}[\widehat{Y}] \quad (4.1)$$

$$= \bar{y} \pm T_{n-1}^* \times \frac{s_y}{\sqrt{n}} \quad (4.2)$$

$$= 180.3 \pm 2.09 \times \frac{31.1}{\sqrt{20}} \quad (4.3)$$

$$= 180.3 \pm 2.09 \times 6.96 \quad (4.4)$$

$$= 180.3 \pm 14.6 \quad (4.5)$$

$$= (165.7, 194.9) \quad (4.6)$$

Here:

- $\text{SE}[\widehat{Y}]$ is the **standard error** or **sampling error** of the estimator $\widehat{Y}$ of the population mean $\bar{Y}$;
- T_{n-1}^* is the quantile of the t -distribution with $n - 1$ degrees of freedom so that the interval $[-T_{n-1}^*, +T_{n-1}^*]$ encloses 95% of the probability of the t -distribution;
- $\text{MOE}[\widehat{Y}]$ is the **margin of error**, the half-width of the confidence interval, and is the standard error scaled by T_{n-1}^* .

This is what is known as a **model based** estimate, the statistical model being

$$y_i = \mu + \varepsilon_i \quad \text{where } \varepsilon_i \stackrel{\text{iid}}{\sim} N(0, \sigma^2) \quad (4.7)$$

There are just two parameters here: the population mean μ , and the population standard deviation σ .

We can fit this simplest of all linear regression models using the `lm()` function and get the same results:

```
lm1 <- lm(height ~ 1, data=samplemembers)
summary(lm1)

##
## Call:
## lm(formula = height ~ 1, data = samplemembers)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -101.30   -6.05    0.70   11.20   53.70
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  180.300      6.961    25.9 2.77e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 31.13 on 19 degrees of freedom
##
## confint(lm1)
##
##              2.5 % 97.5 %
## (Intercept) 165.73 194.87
```

Here (**Intercept**) is the estimate for μ , the population mean.

The ‘residual standard error’ in the R output is actually what we’d rather call $\hat{\sigma}$, our estimate of the standard deviation σ .

The confidence interval is only valid if the model in equation (4.7) holds: i.e. if the data are normally distributed about the mean. The histogram in Figure 4.1 suggests that this is not the case however, since there is a clear outlier at low height.

The Central Limit Theorem (CLT) tells us that if n is large enough then our estimate for μ is useable, and its confidence interval realistic, even if the model in equation (4.7) doesn’t actually hold.

The CLT enables much of statistical practice to yield meaningful results even without our knowing the true data generation model.

In this course most of the examples will have sample sizes large enough that the CLT will mean our results are valid. In these cases when constructing a confidence interval it will be sufficient to replace T_{n-1}^* with the equivalent $Z^* = 1.96$ from the Normal distribution.

4.2 Inference in finite populations

Recall that the standard error of an estimate is the variability of estimates between different samples: in fact it is defined as the standard deviation of all possible estimates from all possible estimates drawn using the same sample design.

In an infinite population there are an infinite number of possible samples.

However in a finite population that isn’t the case. In a population of size N if we draw a simple random sample without replacement of size n there are

$$\binom{N}{n} = {}^N C_n \quad (4.8)$$

$$= \frac{N!}{n!(N-n)!} \quad (4.9)$$

$$= \frac{N \times (N-1) \times (N-2) \times \dots \times (N-n+1)}{n \times (n-1) \times (n-2) \times \dots \times 1} \quad (4.10)$$

possible samples: i.e. the number of ways it is possible to draw n objects from a collection of N without regard to the order they are selected.

In our example of selecting $n = 20$ from $N = 87$ there are an astronomical number of possible samples:

$$\binom{87}{20} = \frac{87!}{20!67!} = 2.38 \times 10^{19}$$

There is of course only one way to select a census though:

$$\binom{87}{87} = \frac{87!}{87!0!} = 1$$

(using the at first face slightly annoying definition $0!=1$). Since there’s only one possible sample when we’re drawing a census the variation among sample estimates from different samples is zero: i.e. the standard error is zero.

It’s also true that if n is close to N then sample estimates hardly differ from one another.

We clearly need a modification of our standard error formula

$$\mathbf{SE}[\widehat{Y}] = \frac{s_y}{\sqrt{n}} \quad (\text{Infinite population})$$

to allow for sample sizes n being close to the population size.

That modification is the **finite population correction (fpc)**:

$$\mathbf{SE}[\widehat{Y}] = \sqrt{1 - \frac{n}{N}} \times \frac{s_y}{\sqrt{n}} \quad (\text{Finite population})$$

The factor $\sqrt{1 - n/N}$ squashes the standard error to zero in the case that $n = N$, but for very small n has a value close to 1, and makes no difference at all.

In our example we had a standard error of

$$\mathbf{SE}[\widehat{Y}] = \frac{s_y}{\sqrt{n}} \quad (4.11)$$

$$= \frac{31.1}{\sqrt{20}} \quad (4.12)$$

$$= 6.96 \quad (4.13)$$

If we incorporate the fpc this becomes

$$\mathbf{SE}[\widehat{Y}] = \sqrt{1 - \frac{n}{N}} \times \frac{s_y}{\sqrt{n}} \quad (4.14)$$

$$= \sqrt{1 - \frac{20}{87}} \frac{31.1}{\sqrt{20}} \quad (4.15)$$

$$= 0.8776 \times \frac{31.1}{\sqrt{20}} \quad (4.16)$$

$$= 6.11 \quad (4.17)$$

This narrows the confidence interval for the estimate of the population mean height from (165.7,194.9) to (167.5,193.1).

This is a **design based** confidence interval, in which it is the sample design which foremost in our derivation of standard errors.

There is quite a lot of theory behind derivation of standard errors in the complex designs which arise in sample surveys.

The **survey** package in R implements the consequences of sample designs which will mean we can use the theory without having to derive it.

For example here's a call to some **survey** package functions which gives us the effect of the fpc in our example:

```
samplemembers$bigN <- bigN
ss <- svymean(height~1, design=svydesign(ids=~1, fpc=~bigN, data=samplemembers))
ss

##           mean      SE
## height 180.3 6.1089

confint(ss)

##           2.5 %    97.5 %
## height 168.3268 192.2732
```


Table 4.1: Sample Frame (N rows)

Unit ID	Auxiliary	Selection	Selection
	Data	Probability	Indicator
1	$\mathbf{X}_1$	π_1	I_1
2	$\mathbf{X}_2$	π_2	I_2
3	$\mathbf{X}_3$	π_3	I_3
$\vdots$	$\vdots$	$\vdots$	$\vdots$
i	$\mathbf{X}_i$	π_i	I_i
$\vdots$	$\vdots$	$\vdots$	$\vdots$
N	$\mathbf{X}_N$	π_N	I_N

If you're interested in the theory take a look at a text like Cochran (1977) or Särndal et al. (1992).

4.3 The survey inference process

Sampling begins with the sample frame - usually this is a **list** of the members, or **units**, which make up the population.

The list contains at minimum some **identifier** of each unit, which enables it to be found/contacted/measured once it has been selected, and possibly a lot of other information too.

Data that we have on the frame is called **auxiliary information**: it is data that is available about a unit whether or not it has been sampled. Some sample designs rely on this auxiliary data during the process of sample selection.

In probabilistic (random) sampling each unit i has a known nonzero chance of selection π_i , which may depend on the auxiliary data in some way. These probabilities don't have to be equal, just known and nonzero.

In random sampling we generate a random variable I_i for each population member, based on the sample design, to decide whether it is selected into the sample or not:

$$I_i = \begin{cases} 1, & \text{if unit } i \text{ is selected} \\ 0, & \text{otherwise} \end{cases} \quad (4.18)$$

The sample size is just the sum of these random variables

$$n = \sum_{i=1}^N I_i$$

So we start with the N rows of the sample frame (Table 4.1) generate the sample indicators I_i , and reduce to the n rows of data in Table 4.2. For each member k of the sample we know its population identifier i_k , its auxiliary information $\mathbf{x}_k$, selection probability π_k , and we also collect and record the survey data $\mathbf{y}_k$.

The **expected value** of each indicator variable is π_i

$$E[I_i] = 1 \times \Pr(I_i = 1) + 0 \times \Pr(I_i = 0) \quad (4.19)$$

$$= 1 \times \pi_i + 0 \times (1 - \pi_i) \quad (4.20)$$

$$= \pi_i \quad (4.21)$$

Table 4.2: Sample Data (n rows)

Sample Unit	Unit ID	Auxiliary Data	Selection Probability	Sample Weight	Sample Data
1	i_1	$\mathbf{x}_1$	π_1	w_1	$\mathbf{y}_1$
2	i_2	$\mathbf{x}_2$	π_2	w_2	$\mathbf{y}_2$
3	i_3	$\mathbf{x}_3$	π_3	w_3	$\mathbf{y}_3$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
k	i_k	$\mathbf{x}_k$	π_k	w_k	$\mathbf{y}_k$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
n	i_n	$\mathbf{x}_n$	π_n	w_n	$\mathbf{y}_n$

Each selected sample member carries a **sample weight** which is the number of units in the population that that unit represents. These weights are defined as the inverses of the selection probabilities.

$$W_i = \frac{1}{\pi_i}$$

Consider some characteristic Y_i which each population member has. e.g. in a survey of households we might be measuring the amount of money Y_i that household i spends on internet streaming services. The total amount of money spent by all households in the population combined is

$$Y = \sum_{i=1}^N Y_i$$

However we only know Y_i for some households: those which are selected, i.e. those which have $I_i = 1$.

The **Horwitz-Thompson** (HT) estimator of the population total is

$$\widehat{Y} = \sum_{i=1}^N I_i W_i Y_i \quad (4.22)$$

$$= \sum_{k=1}^n w_k y_k \quad (4.23)$$

i.e. a weighted sum of the observed household spending values y_k just for the n selected households. ($I_i = 0$ for any unselected households, so the sum of N terms simplifies to a sum of n terms, just over the sample members.)

We can see that the HT estimator is **unbiased**:

$$E[\widehat{Y}] = \sum_{i=1}^N E[I_i] W_i Y_i \quad (4.24)$$

$$= \sum_{i=1}^N \pi_i W_i Y_i \quad (4.25)$$

$$= \sum_{i=1}^N \pi_i \frac{1}{\pi_i} Y_i \quad (4.26)$$

$$= \sum_{i=1}^N Y_i \quad (4.27)$$

$$= Y \quad (4.28)$$

ID	Address	Value (\$M)
6451	1 Chestnut Place	1.201
6452	2 Chestnut Place	0.780
8773	2A Chestnut Place	0.520
7164	3 Chestnut Place	2.530
6125	4 Chestnut Place	0.900

i.e. although the observed value $\widehat{Y}$ from a given sample may not equal the true value Y , when averaged over all possible samples we can see that on average it does equal the true population value.

We also want the standard error $\mathbf{SE}[\widehat{Y}]$ to be small, so that we have precise estimates.

We're all familiar with the idea that the larger the sample size n the smaller the standard error. We now know that the standard error is going to crush down to zero in finite populations the closer we get to a census.

There's a further advantage we can get by making a good choice of **sample design**.

Two different methods of selecting a sample of the **same sample size** n may lead to very different values of the standard error, and thus confidence intervals of very different widths.

Incidentally, the expected value of the sum of the sample weights among sample members is equal to the population size:

$$\sum_{k=1}^n w_k = \sum_{i=1}^N I_i W_i \quad (4.29)$$

$$E \left[\sum_{k=1}^n w_k \right] = \sum_{i=1}^N E[I_i] W_i \quad (4.30)$$

$$= \sum_{i=1}^N \pi_i W_i \quad (4.31)$$

$$= \sum_{i=1}^N \pi_i \frac{1}{\pi_i} \quad (4.32)$$

$$= \sum_{i=1}^N 1 \quad (4.33)$$

$$= N \quad (4.34)$$

4.4 Example

We might have a small street of five houses. Our frame is just the list of addresses. We might know the household rateable value from the City Council list of dwellings.

We might then select a simple random sample of $n = 2$ houses from the $N = 5$ in the small street. The probability of selection is $\pi_i = 2/5$ for each house. We draw our random sample and select house number 2A and house number 5 into our sample.

We visit each selected house and ask how much they spend y_k on all their internet streaming services each month.

Each house in the sample has a weight of $2.5 = 5/2$: i.e. it represents itself and 1.5 other houses in the street. The sum of the weights of the two selected houses is equal to the population size $N = 5$.

ID	Address	Value (\$M)	Prob. π_i	Selected, I_i
6451	1 Chestnut Place	1.201	0.4	0
6452	2 Chestnut Place	0.780	0.4	0
8773	2A Chestnut Place	0.520	0.4	1
7164	3 Chestnut Place	2.530	0.4	0
6125	4 Chestnut Place	0.900	0.4	1

	ID	Address	Value (\$M)	Weight w_k	Internet spend, y_k
3	8773	2A Chestnut Place	0.52	2.5	75
5	6125	4 Chestnut Place	0.90	2.5	19

Our sample is really too small to do any sensible inference on: but let's do it anyway to show how things work. We calculate the mean and standard deviation of the monthly internet streaming values: $\bar{y} = \$47.00$, and $s_y = \$39.60$.

Our estimate of the mean monthly spend is $\bar{y} = \$47.0$, with a margin of error of

$$\mathbf{MOE}[\widehat{Y}] = Z^* \sqrt{1 - \frac{n}{N}} \times \frac{s_y}{\sqrt{n}} \quad (4.35)$$

$$= 1.96 \sqrt{1 - \frac{2}{5}} \times \frac{\$39.60}{\sqrt{2}} \quad (4.36)$$

$$= \$42.51 \quad (4.37)$$

which gives a 95% confidence interval of (\$4.49,\$89.51) (although the use of the large sample value $Z^* = 1.96$ is very much **not** justified here.)

In finite populations we are sometimes interested in population totals - which don't make any sense in infinite populations. Now we have an estimate of the mean spend per house, we can scale it up by the population size to see how much per month is being spent by all of the houses combined:

$$\widehat{Y} = N\widehat{Y} \quad (4.38)$$

$$= 5 \times \$47.00 \quad (4.39)$$

$$= \$235.00 \quad (4.40)$$

and we can scale the confidence interval up in the same way

$$\mathbf{CI}[\widehat{Y}] = N\mathbf{CI}[\widehat{Y}] \quad (4.41)$$

$$= 5 \times (\$4.49, \$89.51) \quad (4.42)$$

$$= (\$22.45, \$447.55) \quad (4.43)$$

Chapter 5

Sample Designs

In this Chapter we will consider the various sample designs in common use, the reasons for using each one, and methods for selecting from suitable frames.

We'll use a variety of datasets.

5.1 Census

The simplest of all! We select every unit in the population, so $n = N$.

The probability of selection is

$$\pi = \frac{n}{N} = \frac{N}{N} = 1$$

i.e. every unit is selected with certainty, and the sample weights are all

$$w_i = \frac{1}{\pi_i} = \frac{1}{1} = 1$$

so each unit in the sample represents itself alone.

Design considerations

None: we're sampling all N members of the population.

5.2 Simple Random Sampling Without Replacement (SRSWOR)

The Simple Random Sample WithOut Replacement is the most fundamental of all sample designs, and it appears nested inside many more complex designs as we'll see later.

We have a population of size N which is fully enumerated (i.e. fully listed), and we want to select a sample of size n .

The defining property of a SRSWOR is that all samples of size n have an equal chance of selection. A **consequence** of this definition is that all population units have the same chance of selection.

There are

$$\binom{N}{n} = {}^N C_n = \frac{N!}{n!(N-n)!}$$



possible samples. The R function `choose(n,k)` can be used for evaluating the number of possible samples. e.g. the number of ways to select $n = 20$ units from $N = 87$ is

```
choose(87,20)
```

```
## [1] 2.375456e+19
```

The probability of selection of each unit is

$$\pi_i = \frac{n}{N}$$

leading to sample weights

$$w_i = \frac{N}{n}$$

The SRSWOR is used when the **costs** of different samples are not known to differ (e.g. there are no units that are more costly to visit than others), and where there is no informative auxiliary information.

If we have a sample frame stored as an R data frame `sframe` with a column `ID` uniquely identifying the units, then we can select an SRSWOR as follows:

```
bigN <- nrow(sframe)
samp <- sframe[sample(bigN,n),]
samp$weight <- bigN/n
```

or

```
bigN <- nrow(sframe)
IDset <- sample(sframe$ID,n)
samp <- sframe[sframe$ID %in% IDset,]
samp$weight <- bigN/n
```

or as a function

```
select.srswor <- function(df, n) {
  # SRSWOR
  bigN <- nrow(df)
  if(n <= bigN) {
    samp <- df[sample(bigN,n,replace=FALSE),]
    samp$bigN <- bigN
    samp$weight <- bigN/n
  } else {
    stop("Cannot have a SRSWOR with n>N")
  }
  return(samp)
}
```

Design considerations

- We only need to decide on the sample size n (see later).

Example

Select $n = 5$ units from a frame listing of airports with IATA codes sourced from Kaggle <https://www.kaggle.com/datasets/syedasimalishah/airports/data>

The top ten rows of the dataset of $N = 6072$ are shown here:

Sir



ID	IATA	Name	City	Country	Latitude	Longitude
1	AAA	Anaa Airport	Anaa	French Polynesia	-17.352600	-145.509995
2	AAC	El Arish International Airport	El Arish	Egypt	31.073299	33.835800
3	AAE	Rabah Bitat Airport	Annaba	Algeria	36.822201	7.809174
4	AAF	Apalachicola Regional Airport	Apalachicola	United States	29.727501	-85.027496
5	AAH	Aachen-Merzbrück Airport	Aachen	Germany	50.823055	6.186389
6	AAK	Buariki Airport	Buariki	Kiribati	0.185278	173.636993
7	AAL	Aalborg Airport	Aalborg	Denmark	57.092759	9.849243
8	AAM	Malamala Airport	Malamala	South Africa	-24.818100	31.544600
9	AAN	Al Ain International Airport	Al Ain	United Arab Emirates	24.261700	55.609200
10	AAO	Anaco Airport	Anaco	Venezuela	9.430225	-64.470726

Table 5.1: Sample of airports (SRSWOR)

ID	IATA	Name	City	Country	Latitude
3593	NTD	Point Mugu Naval Air Station (Naval Base Ventura Co)	Point Mugu	United States	34.12030
1472	EUM	Neumünster Airport	Neumuenster	Germany	54.07944
2512	KHZ	Kauehi Airport	Kauehi	French Polynesia	-15.78080
197	ANC	Ted Stevens Anchorage International Airport	Anchorage	United States	61.17440
4710	SRN	Strahan Airport	Strahan	Australia	-42.15500

And here is a SRSWOR of $n = 5$ airports

```
samp <- airports |> select.srswor(n=5)
```

5.3 Simple Random Sampling With Replacement (SRSWR)

This design is rarely used, but is the simplest form of Probability Proportional to Size With Replacement (PPSWR) sampling, but with the sizes all equal.

Sampling proceeds in principle by selecting one unit at a time where each unit has a $1/N$ chance of being selected each time. After n draws the probability a unit has been selected at least once is

$$p_i = 1 - \left(1 - \frac{1}{N}\right)^n$$

(This is 1 minus the chance that a unit is not selected on each of the n sampling occasions.)

If n is tiny compared to N , p_i is approximately n/N which is the SRSWOR selection probability. In practice we assign a weight of N/n to each unit as it is selected, so a unit that is selected m times ends up with a weight of mN/n .

If we use weights calculated in this way we have what is known as the Hansen-Hurwitz estimator, rather than the Horwitz-Thompson estimator. However this distinction need not concern us.

Design considerations

As in a SRSWOR, we only need to decide on the sample size n .

Table 5.2: Sample of airports (SRSWOR)

ID	IATA	Name	City	Country	Latitude	Longitude	nsampled
805	CCB	Cable Airport	Upland	United States	34.11160	-117.6880	1
1063	CUD	Caloundra Airport	Caloundra	Australia	-26.80000	153.1000	1
1077	CVE	Coveñas Airport	Coveñas	Colombia	9.40092	-75.6913	1
2738	LAY	Ladysmith Airport	Ladysmith	South Africa	-28.58170	29.7497	1
4804	SXN	Sua Pan Airport	Sowa	Botswana	-20.55340	26.1158	1

Example

Here is a SRSWR of $n = 5$ airports: note that we have aggregated across any instances of multiple selections, so that in the output sample list each unit appears only once no matter how often it was selected.

```
bigN <- nrow(airports)
n <- 5
samp <- airports[sample(bigN,n,replace=TRUE),]
samp$weight <- bigN/n
samp <- samp %>%
  group_by(ID) %>%
  summarise(across(-weight, first), nsampled=n(), weight=sum(weight))
```

or as a function

```
select.srswr <- function(df, n) {
  # SRSWR
  bigN <- nrow(df)
  df$rownames <- 1:bigN
  samp <- df[sample(bigN,n,replace=TRUE),]
  samp$bigN <- bigN
  samp$weight <- bigN/n
  samp <- samp %>%
    group_by(rownames) %>%
    summarise(across(-weight, first), nsampled=n(), weight=sum(weight)) |>
    as.data.frame() |>
    ungroup() |>
    select(-rownames)
  return(samp)
}

samp <- airports |> select.srswr(n=5)
```

5.4 Probability proportion to size Sampling (PPSWR)

This sample design generalises the SRSWR design to unequal probabilities of selection. This design recognises that some units may have more influence on our estimates because of some **size** property that is known (it must be on the frame). We therefore give these units higher probabilities of being selected, and therefore lower weight.

We draw n units, one at a time, but with probabilities which are proportional to the size measure X_i : thus the probability of selection of unit i on each of the n draws from the

Country	Code	Year	Population	Prob
Afghanistan	AFG	2023	41454762	0.0051249
Albania	ALB	2023	2811660	0.0003476
Algeria	DZA	2023	46164222	0.0057071
American Samoa	ASM	2023	47544	0.0000059
Andorra	AND	2023	80869	0.0000100
Angola	AGO	2023	36749909	0.0045432
Anguilla	AIA	2023	14435	0.0000018
Antigua and Barbuda	ATG	2023	93331	0.0000115
Argentina	ARG	2023	45538402	0.0056297
Armenia	ARM	2023	2943398	0.0003639

population is

$$p_i = \frac{X_i}{\sum_{j=1}^N X_j}$$

Approximating the probability of selection π after n draws as np_i leads to the sample weights

$$w_i = \frac{1}{np_i}$$

In a PPSWR sample these weights are not guaranteed to add up to the population size.

Design considerations

We need to decide on:

- the size variable X_i (which must be known for every unit in the population), and
- the sample size n .

The size variable X_i should correlate in some way with the population characteristics we want to estimate. If we're interested in estimating the total economic output of businesses, then the amount of tax each business paid last year would be a good size variable. But it wouldn't be helpful if we were interested in estimating the proportion of businesses with family friendly employment practices.

With replacement designs lose some efficiency because their estimators do not incorporate a finite population correction in their standard errors. For small sampling fractions (i.e. small n/N) this is not a significant drawback.

Example

We have a list of $N = 236$ countries and their 2023 populations: the first 10 are listed below

```
pop[1:10,] %>%
  kbl() %>%
  kable_styling()
```

Select a PPSWR sample of $n = 10$ countries, with probability proportional to their population sizes.

```
set.seed(111)
bigN <- nrow(pop)
n <- 10
samp <- pop[sample(bigN,n,prob=pop$Prob,replace=TRUE),]
```

```
samp$weight <- 1/(n*samp$Prob)
samp <- samp %>%
  group_by(Country) %>%
  summarise(across(-weight, first), nsampled=n(), weight=sum(weight))
```

or as a function

```
select.ppswr <- function(df, n, sizevar) {
  # PPSWR
  bigN <- nrow(df)
  df$rownames <- 1:bigN
  samp <- df[sample(bigN,n,prob=df[,sizevar],replace=TRUE),]
  samp$bigN <- bigN
  samp$weight <- bigN/n
  samp <- samp %>%
    group_by(rownames) %>%
    summarise(across(-weight, first), nsampled=n(), weight=sum(weight)) |>
    as.data.frame() |>
    ungroup() |>
    select(-rownames)
  return(samp)
}
```

```
samp <- pop |> select.ppswr(n=10, sizevar="Population")
samp
```

##	Country	Code	Year	Population	Prob	bigN	nsampled	weight
## 1	Bangladesh	BGD	2023	171466986	0.021197666	236	1	23.6
## 2	Brazil	BRA	2023	211140731	0.026102347	236	1	23.6
## 3	India	IND	2023	1438069597	0.177781857	236	2	47.2
## 4	Indonesia	IDN	2023	281190068	0.034762221	236	1	23.6
## 5	Japan	JPN	2023	124370947	0.015375402	236	1	23.6
## 6	Pakistan	PAK	2023	247504505	0.030597831	236	1	23.6
## 7	South Africa	ZAF	2023	63212386	0.007814653	236	1	23.6
## 8	United States	USA	2023	343477330	0.042462505	236	2	47.2

5.5 Linear Systematic Random Sample (LSRS)

This design is often used in cases where the population is not fully enumerated, but is presented to us in the form of a queue. For example we might want to select one in ten passengers cars checking in for a ferry crossing. Another example is where there is information in the ordering of a list: for example a list might be sorted from youngest to oldest, or sorted from north to south. We can space out our sample down the list, avoiding the chance that our sample doesn't have a good spread of ages, or a good spread of latitudes.

We select n units from N by first calculating the **skip** k , which is the nearest integer at or below N/n . We then choose at random one unit from the first k of our list or queue, say that is unit number j . We then select units $j, j+k, j+2k, j+3k$, etc.

Selecting one unit at random in the first k (rather than just selecting the **first** unit, as we might naively think of doing) means that every unit in the list has a chance of selection.

The selection probabilities are

$$\pi_i = \frac{n}{N}$$

and weights are

$$w_i = \frac{N}{n}$$

Data from this design are often analysed as if they were drawn as a SRSWOR, but in fact this is a special case of a one-stage cluster sample (S1C, see below).

Note that if there are N units but N is not an exact multiple of the skip k , then the sample size will differ depending on which of the first k units we select to initiate sampling. For example if we wanted to select $n = 3$ units by LSRS from a list of $N = 11$ units, then the skip size is $k = \text{floor}(11/3) = \text{floor}(3.67) = 3$. If we start the process by selecting the first unit, we'd end up with the sample of 4 units, $\{1, 4, 7, 10\}$, but if we start by selecting the 4th unit we'd end up with only 3 units, $\{4, 7, 10\}$.

Another common reason to take an LSRS is in household surveys: we're selecting dwellings from a small area, and for reasons of confidentiality (we don't want members of different selected households to be aware that they are both in the same survey), or to prevent collusion between neighbouring selected households, we space the sample out along the street using a LSRS to separate the selected dwellings as much as possible.

An LSRS is also a convenient way to sample if you have a printed list.

After setting your random starts location you can skip down the list at fixed intervals.

5.5.1 Design considerations

- Choose a sample size n
- Consider the ordering of the list and whether units differ systematically through the list

An LSRS has one risk, and that is a periodicity in the frame that you are unaware of. For example if we are selecting apartments from a building to inspect for structural weaknesses, and select every fourth one (after a random start), we had better be sure that there aren't exactly four apartments per floor: otherwise we'll have inadvertently selected a column of apartments stacked one above the other, e.g. all in the north-east corner. We'll miss any systematic issues that occur with the north-west apartments in that case.

Such unknown periodicities are unlikely in most cases, but we should be aware that they can occur.

Example

To select a LSRS of airports, let's order the airports by latitude, and then take a LSRS of $n = 5$ units.

```
bigN <- nrow(airports)
airports <- airports[order(airports$Latitude),]
n <- 5
k <- floor(bigN/n)
cat("Skip size is k =", k, "\n")
```

```
## Skip size is k = 1214
```

```
i0 <- sample(k, 1) # first selected unit
cat("Starting sampling at unit", i0, "\n")
```

```
## Starting sampling at unit 305
```

Linear



Table 5.3: Sample of airports (SRSWOR)

	ID	IATA	Name	City	Country	Latitude	Longitude
5304	5304	VAO	Suavanao Airport	Suavanao	Solomon Islands	-7.58556	158.73100
4746	4746	STV	Surat Airport	Surat	India	21.11410	72.74180
1714	1714	GIB	Gibraltar Airport	Gibraltar	Gibraltar	36.15120	-5.34966
1638	1638	FYN	Fuyun Koktokay Airport	Fuyun	China	46.80417	89.51201
5277	5277	UTO	Indian Mountain LRRS Airport	Indian Mountains	United States	65.99280	-153.70399

```

n <- (bigN-i0)%/%k + 1 # recalculate the sample size
cat("Actual sample size is",n,"\n")

## Actual sample size is 5

samp <- airports[i0 + (0:(n-1))*k,]
samp$weight <- bigN/n

select.lsr <- function(df, n) {
  # LSR
  bigN <- nrow(df)
  k <- floor(bigN/n)
  i0 <- sample(k,1) # first selected unit
  n <- (bigN-i0)%/%k + 1 # recalculate the sample size
  samp <- df[i0 + (0:(n-1))*k,]
  samp$bigN <- bigN
  samp$weight <- bigN/n
  return(samp)
}

samp <- airports %>% select.lsr(5)

```

5.6 Stratified Simple Random Sample (STSRS)

In this design we take our population of N units and break it into H subgroups using our auxiliary data from the frame. We then draw separate samples from **each of the subgroups**.

To draw a stratified sample each population unit needs to be assigned in advance to one, and only one, of these subgroups - which are known as **strata** (singular: **stratum**).

There are N_h units in stratum h , and all of these stratum-level population sizes add to the total population size:

$$N = \sum_{h=1}^H N_h$$

Dividing both sides of this equation by N we have the stratum fractions $F_h = N_h/N$: these are the proportions of the **population** that belong to the various strata, and they add up to 1:

$$1 = \sum_{h=1}^H \frac{N_h}{N} = \sum_{h=1}^H F_h$$

For example, we can assign all residential addresses in New Zealand to exactly one of a defined set of non-overlapping regions, such as regional council areas. Or we might just have

two strata: North Island, South Island (neglecting any addresses on offshore islands). There are \$N=2.4\$ million addresses in New Zealand, of which $N_1 = 1.72$ million in the North Island ($F_1 = 0.74 = 74$) and $N_2 = 0.62$ million are in the South Island ($F_2 = 0.26 = 26$).

We then divide or **allocate** our total sample size n to the strata. Stratum h is allocated n_h units, and the total sample size is then the sum of these:

$$n = \sum_{h=1}^H n_h$$

We then carry out independent sampling in each of the strata.

The way we choose to define the strata and the way we then choose to allocate the sample across the strata depend on our goals.

Stratified sampling is done for a number of reasons

1. **Estimation Efficiency** - efficiency is the word statisticians use when considering how we can get standard errors which are as **small as possible**.

In business surveys we often stratify because businesses differ from each other dramatically, particularly in size measures such as turnover or size of workforce. That means a small number of businesses (e.g. Spark, Air New Zealand, Fonterra) have a very large effect on estimates, and they differ dramatically from each other as well. If they happen to be included or happen to be missed from our samples that means we'd get estimates which differ very much between samples. To protect ourselves against this we stratify by business size (using tax data). The largest businesses are in the top stratum, which is small, but a census is always done of those businesses (the top stratum is called a **full coverage stratum**).

The smallest businesses are in the lowest stratum, and they are selected with low probability: since they resemble each other more, and one can stand in for another.

Having a strata designed so that the units within the strata resemble each other strongly (one can stand in for another easily), but where units in different strata differ strongly from each other. For the same sample size as a SRSWOR stratified designs can sometimes achieve much much smaller standard errors.

2. **Subgroup estimates** - we may stratify by region because we want to make estimates for each region separately. In that case we need to ensure that our sample is large enough in each stratum to get reliable estimates. A national SRSWOR runs the risk that one region might happen to miss out on getting much sample.
3. **Operational considerations** - we may stratify by region because that is the easiest way to manage an interview workforce - we may have interviewers based in a number of regional cities, and having them interview in their local areas is the best way to organise that workforce.

Two common methods for allocating the n sample units across the H strata are:

1. **Equal allocation**

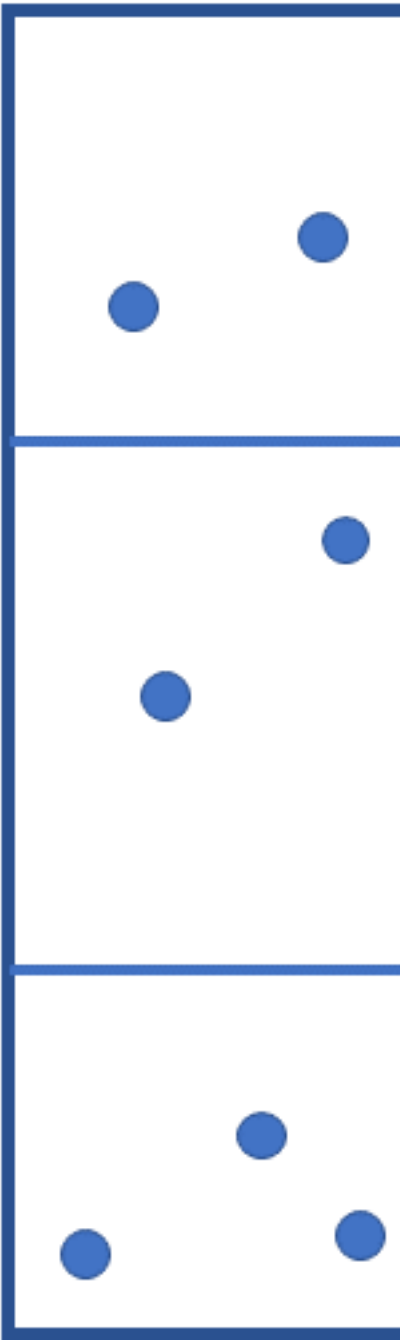
$$n_h = \frac{n}{H}$$

Every stratum gets the same number of units allocated - independent of the stratum size. That means we get different weights in each stratum:

$$w_h = \frac{N_h}{n_h} = H \frac{N_h}{n}$$

This is sensible when we want to have high precision estimates in each stratum separately.

Stratified S



h	Education Region	N_h	F_h
1	Bay of Plenty, Waiairiki	185	0.0739
2	Canterbury, Chatham Islands	283	0.1130
3	Hawke's Bay, Tairāwhiti	174	0.0695
4	Nelson, Marlborough, West Coast	127	0.0507
5	Otago, Southland	228	0.0911
6	Tai Tokerau	151	0.0603
7	Taranaki, Whanganui, Manawatu	234	0.0935
8	Tāmaki Herenga Manawa	188	0.0751
9	Tāmaki Herenga Tāngata	197	0.0787
10	Tāmaki Herenga Waka	179	0.0715
11	Waikato	277	0.1106
12	Wellington	281	0.1122

2. Proportional allocation

$$n_h = nF_h = n \frac{N_h}{N}$$

Here large strata get a lot of units allocated to them, and small strata get fewer units. The consequence of this design is that the weights of all units are equal, no matter which stratum they come from:

$$w_h = \frac{N_h}{n_h} = \frac{N}{n}$$

This allocation is also called **self weighting**, and the weights are the same as in SRSWOR. (Though note: even though the weights are the same, the standard errors of estimates can differ **very significantly** from those from a SRSWOR.)

Design considerations

- Choose the stratification variable(s) X_i , decide the number of strata H , define the strata using the stratification variable(s), determine the stratum sizes N_h , and stratum fractions $F_h = N_h/N$.
- Choose the overall sample size n and **allocate** it across the strata.

Example

The 2026 directory of NZ schools available from <https://www.educationcounts.govt.nz/directories/list-of-nz-schools> lists 2504 schools of various types. Each School is associated with one of $H = 12$ Education Regions, with counts N_h and stratum fractions F_h listed below.

We can select a stratified sample using the following function which implements an independent simple random sample within each stratum with either equal or proportional allocation.

It implements PPSWR sampling within strata as well.

```
select.stsrs <- function(df, n, stratumvar,
                        allocation="equal",
                        method="SRSWOR", sizevar=NULL) {
  bigN <- nrow(df)
  if(!stratumvar%in%names(df)) {
    stop("Stratum variable is not present in the data frame")
  }
  stratumproperties <- as.data.frame(table(df[,stratumvar]))
```

```

names(stratumproperties) <- c(stratumvar,"Nh")
if(allocation=="equal") {
  stratumproperties$nh <- round(n/nrow(stratumproperties))
} else if(allocation=="proportional") {
  stratumproperties$nh <- round(n*stratumproperties$Nh/sum(stratumproperties$Nh))
} else {
  stop("Invalid allocation")
}
df <- merge(df,stratumproperties,by=stratumvar)

if(method=="SRSWOR") {
  samp <- do.call(rbind, by(df, df[,stratumvar],
                           function(dfx) select.srswor(dfx,first(dfx$nh))))
} else if(method=="PPSWR") {
  samp <- do.call(rbind, by(df, df[,stratumvar],
                           function(dfx) select.ppswr(dfx,first(dfx$nh),sizevar=sizevar)))
} else {
  stop("Sampling method invalid")
}
return(samp)
}

```

SRSWOR with Equal allocation

```

samp <- schools |> select.stsrs(n=120,
                              stratumvar="Education_Region", allocation="equal")
as.data.frame(table(samp$Education_Region)) |>
  rename(Education_Region=Var1, SampleSize=Freq)

```

```

##           Education_Region SampleSize
## 1      Bay of Plenty, Waiairiki      10
## 2      Canterbury, Chatham Islands    10
## 3      Hawke's Bay, Tairāwhiti      10
## 4      Nelson, Marlborough, West Coast 10
## 5      Otago, Southland              10
## 6      Tai Tokerau                  10
## 7      Tāmaki Herenga Manawa         10
## 8      Tāmaki Herenga Tāngata        10
## 9      Tāmaki Herenga Waka           10
## 10     Taranaki, Whanganui, Manawatū  10
## 11     Waikato                      10
## 12     Wellington                   10

```

SRSWOR with proportional allocation

```

samp <- schools |> select.stsrs(n=120,
                              stratumvar="Education_Region",
                              allocation="proportional", sizevar="Total")
as.data.frame(table(samp$Education_Region)) |>
  rename(Education_Region=Var1, SampleSize=Freq)

```

```

##           Education_Region SampleSize
## 1      Bay of Plenty, Waiairiki      9
## 2      Canterbury, Chatham Islands   14

```

## 3	Hawke's Bay, Tairāwhiti	8
## 4	Nelson, Marlborough, West Coast	6
## 5	Otago, Southland	11
## 6	Tai Tokerau	7
## 7	Tāmaki Herenga Manawa	9
## 8	Tāmaki Herenga Tāngata	9
## 9	Tāmaki Herenga Waka	9
## 10	Taranaki, Whanganui, Manawatū	11
## 11	Waikato	13
## 12	Wellington	13

PPSWR with Equal Allocation

```
samp <- schools |> select.stsrs(n=120,
                               stratumvar="Education_Region",
                               allocation="equal",
                               method="PPSWR", sizevar="Total")
as.data.frame(table(samp$Education_Region)) |>
  rename(Education_Region=Var1, SampleSize=Freq)
```

##	Education_Region	SampleSize
## 1	Bay of Plenty, Waairiki	9
## 2	Canterbury, Chatham Islands	10
## 3	Hawke's Bay, Tairāwhiti	10
## 4	Nelson, Marlborough, West Coast	8
## 5	Otago, Southland	10
## 6	Tai Tokerau	9
## 7	Tāmaki Herenga Manawa	10
## 8	Tāmaki Herenga Tāngata	9
## 9	Tāmaki Herenga Waka	10
## 10	Taranaki, Whanganui, Manawatū	9
## 11	Waikato	10
## 12	Wellington	10

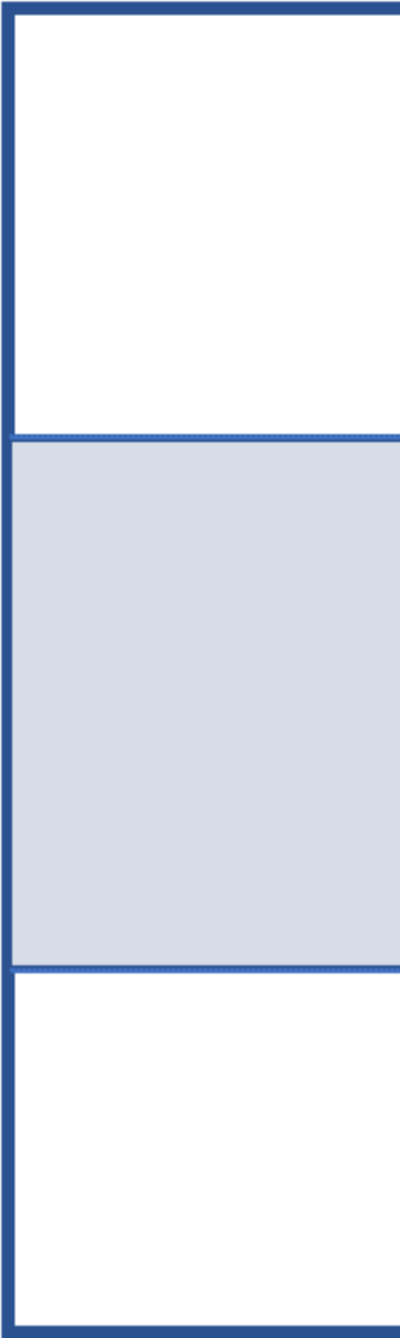
5.7 One stage cluster sampling (S1C)

5.8 Two stage cluster sampling (S2C)

5.9 Complex multistage designs

5.10 Introduction to the design effect

One-sta



Tw



Chapter 6

Inference with R

Data display and inference

Analytic and jackknife methods

- SRSWOR, SRSWR
- Stratified designs
- Cluster designs
- PPSWR
- Complex designs

Chapter 7

Missing data

Origin of missing data

Calibration for unit non-response

- Poststratification

Imputation for item non-response

References

Appendix A

Computing the Required Sample Size

1. Identify your **design estimate** – this is the estimate that will underlie a headline you could imagine being written about your survey.
2. Identify the desired MOE, m , for this design estimate, noting whether you want this MOE to apply to the overall estimate, or for that estimate in each of, say, H subgroups/strata.
3. Compute the sample size required, n_1 , assuming that you're going to select a Simple Random Sample and ignoring the fpc.

For estimation of a mean $\bar{Y}$ of some characteristic y :

$$n_1 = \left(\frac{Z}{MOE} \right)^2 s^2$$

where Z is appropriate for the desired level of confidence and s is the estimated population standard deviation of the characteristic y .

For estimation of a proportion P :

$$n_1 = \left(\frac{Z}{MOE} \right)^2 p(1-p)$$

where p is the estimated proportion, or 0.5 if really unknown.

4. If the sample size is a substantial proportion of the population or subgroup size N , then apply the fpc adjustment:

$$n_2 = \frac{n_1}{1 + \frac{n_1}{N}}$$

5. If you have H subgroups and are using equal allocation then multiply by H

$$n_3 = Hn_2$$

or alternatively sum over the n_2 values in each of the strata you've defined.

6. Apply the design effect (Deff) appropriate for the actual design you'll be using.

$$n_4 = \text{Deff} \times n_3$$

7. Adjust for anticipated non-response: if response rate ϕ is expected then

$$n_5 = \frac{n_4}{\phi}$$

(If non-response is likely to vary between strata, then apply this correction to n_2 , within each stratum.)

Bibliography

- Cochran, W. G. (1977). *Sampling Techniques*. Wiley, New York, third edition.
- Kish, L. (1965). *Survey Sampling*. Wiley, New York.
- Little, R. J. A. and Rubin, D. B. (2002). *Statistical Analysis with Missing Data*. Wiley, Hoboken, second edition.
- Lohr, S. L. (1999). *Sampling: Design and Analysis*. Duxbury Press, Pacific Grove, CA, USA.
- Lumley, T. (2004). Analysis of complex survey samples. *Journal of Statistical Software*, 9(1):1–19. R package version 2.2.
- Lumley, T. S. (2010). *Complex Surveys: A Guide to Analysis Using R (Wiley Series in Survey Methodology)*. Wiley.
- Särndal, C.-E., Swensson, B., and Wretmann, J. (1992). *Model Assisted Survey Sampling*. Springer-Verlag, New York.
- Scheaffer, R. L., III, W. M., and Ott, R. L. (2006). *Elementary Survey Sampling*. Duxbury, Belmont, sixth edition.
- Zealand, S. N. (1995). *A Guide to Good Survey Design*. Statistics New Zealand, Wellington, second edition.